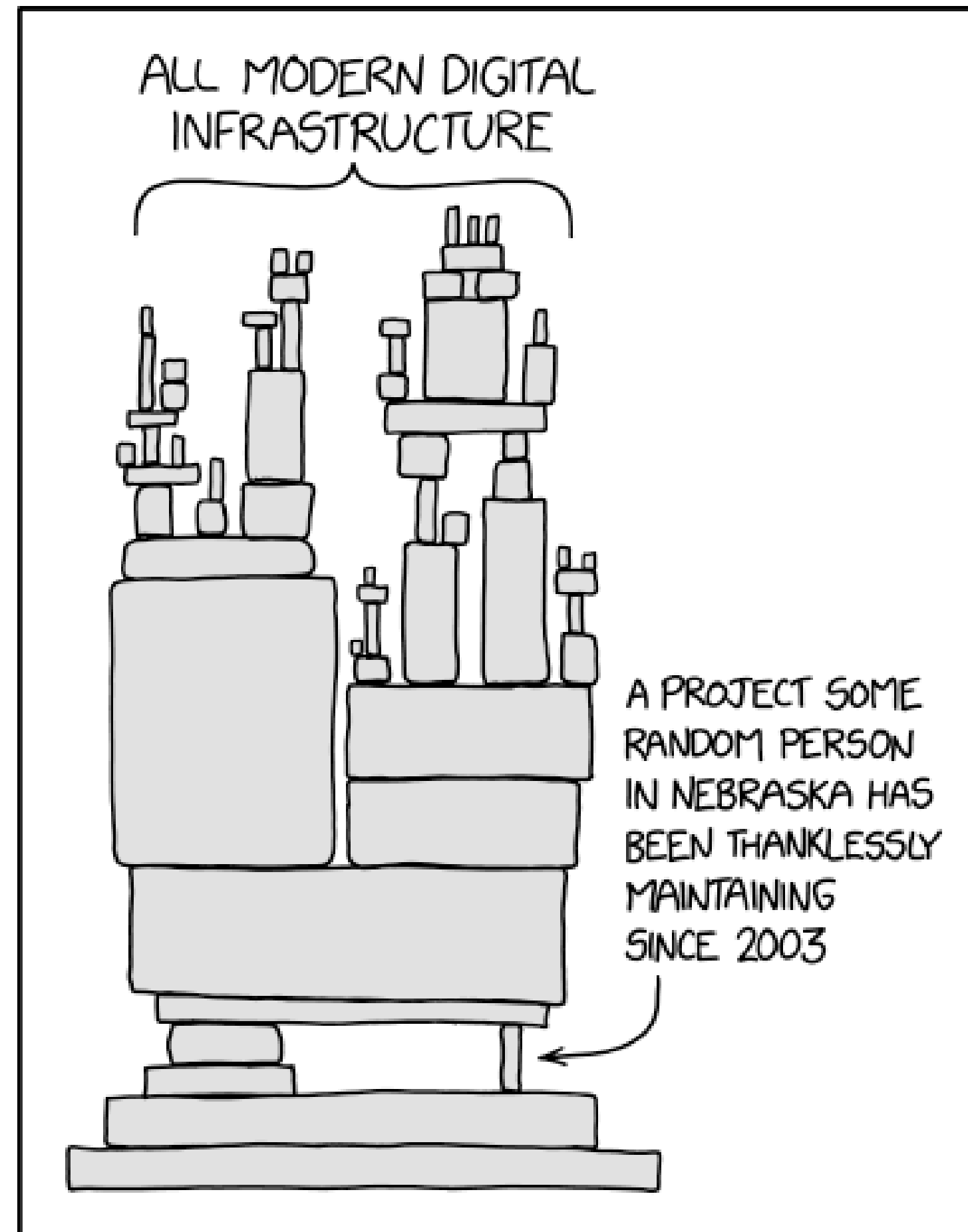


# The Application Layer



Randall Munroe  
<https://xkcd.com/908/>

# Learning Goals

- ▶ Get an understanding of network applications are
  - ▶ Types of network applications
  - ▶ Client-Server Paradigm
  - ▶ Peer-to-peer architectures
  - ▶ Sockets & Process Communication
- ▶ Understand what a application layer protocol is
  - ▶ Properties
  - ▶ Examples

# Learning Goals

- ▶ Get an understanding of HTTP
  - ▶ Basic principles
  - ▶ Persistent & Non-persistent
  - ▶ Requests, responses & their formats
  - ▶ Maintaining state in the stateless HTTP
  - ▶ User tracking with cookies
  - ▶ HTTP/2
  - ▶ Web caching and HTTP performance

# Learning Goals

- ▶ Get a first understanding of e-mail
  - ▶ Terminology
  - ▶ The SMTP protocol
  - ▶ Retrieving e-mails
  - ▶ Securing emails with OpenPGP & S/MIME
- ▶ Get an understanding of the Domain Name System
  - ▶ The DNS hierarchy and types of servers
  - ▶ Iterated & recursive queries
  - ▶ Caching
  - ▶ DNS protocol messages
  - ▶ DNS Security

# Learning Goals

- ▶ Get an understanding of how DNS can help against phishing
  - ▶ Sender Policy Framework
  - ▶ DomainKeys Identified Mail
  - ▶ Domain-based Message Authentication, Reporting and Conformance

Learning Goal FC2A76

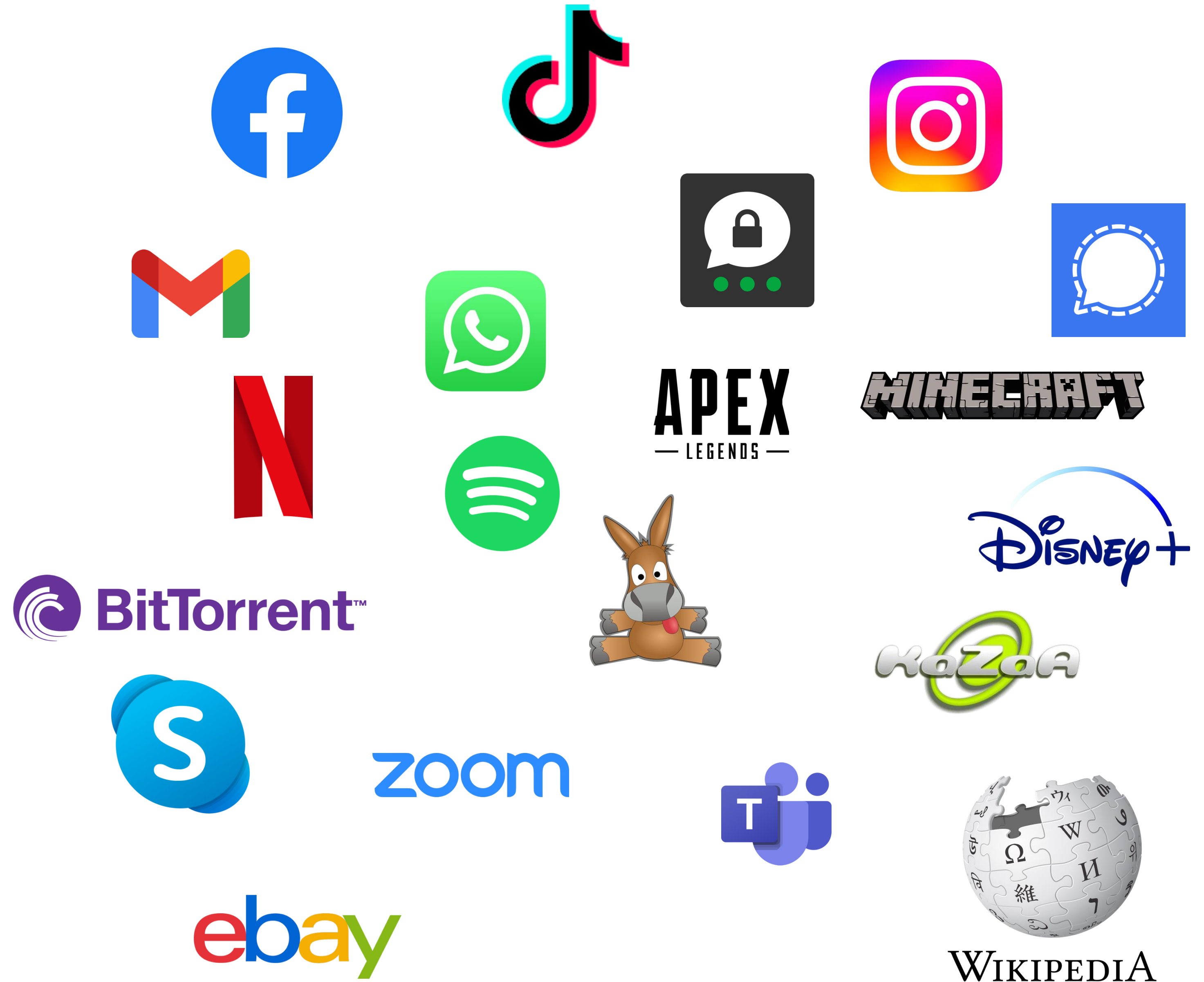
# What are Network Applications?

What are Network Applications?

# Network Applications

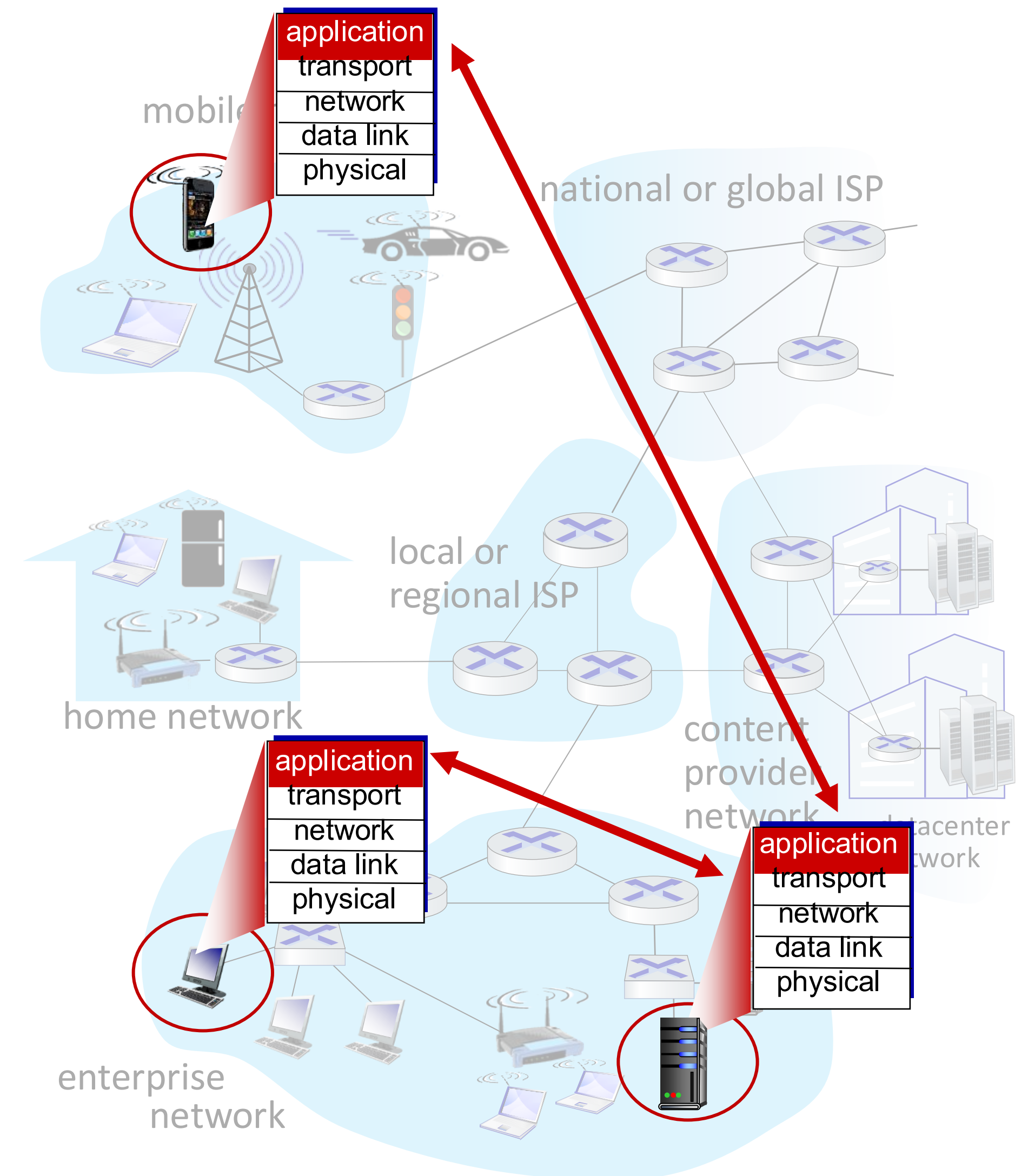
# Network Applications

- ▶ Social Networking
- ▶ Text Messaging
- ▶ E-Mail
- ▶ Games
- ▶ Music & Video Streaming
- ▶ P2P File Sharing
- ▶ Voice over IP
- ▶ Video conferencing
- ▶ Web Services
- ▶ ...



# Creating Network Applications

- ▶ Write programs that
  - ▶ Run on (different) end systems
  - ▶ Communicate over the network
- ▶ No need to write software for network core devices
  - ▶ Network-core devices do not run user applications
  - ▶ Applications on end systems allows for rapid app development, propagation



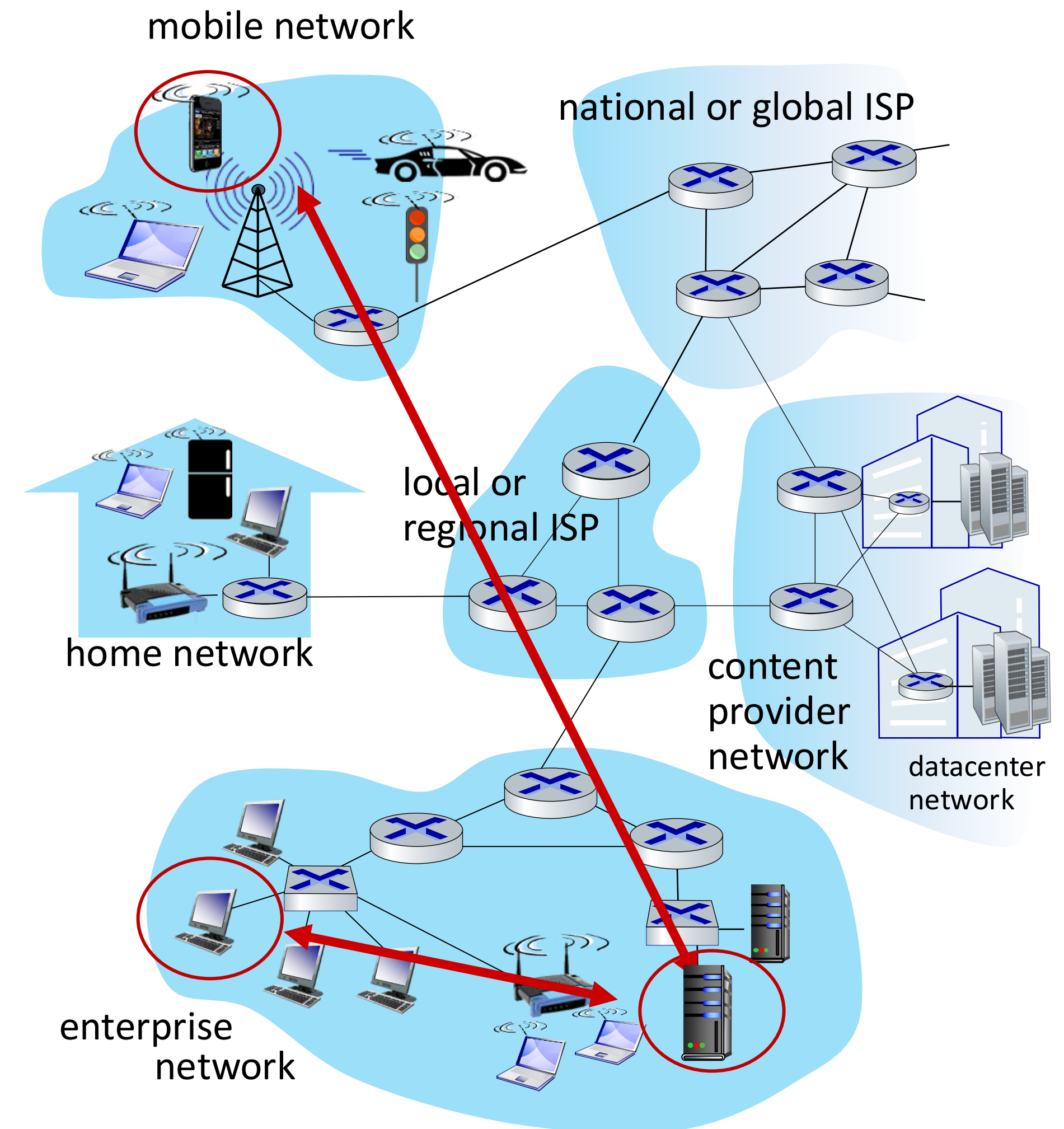
# Client-Server Architecture

## ► Server

- Always-on host
- Permanent (IP) address
- Often in data centres, for scaling

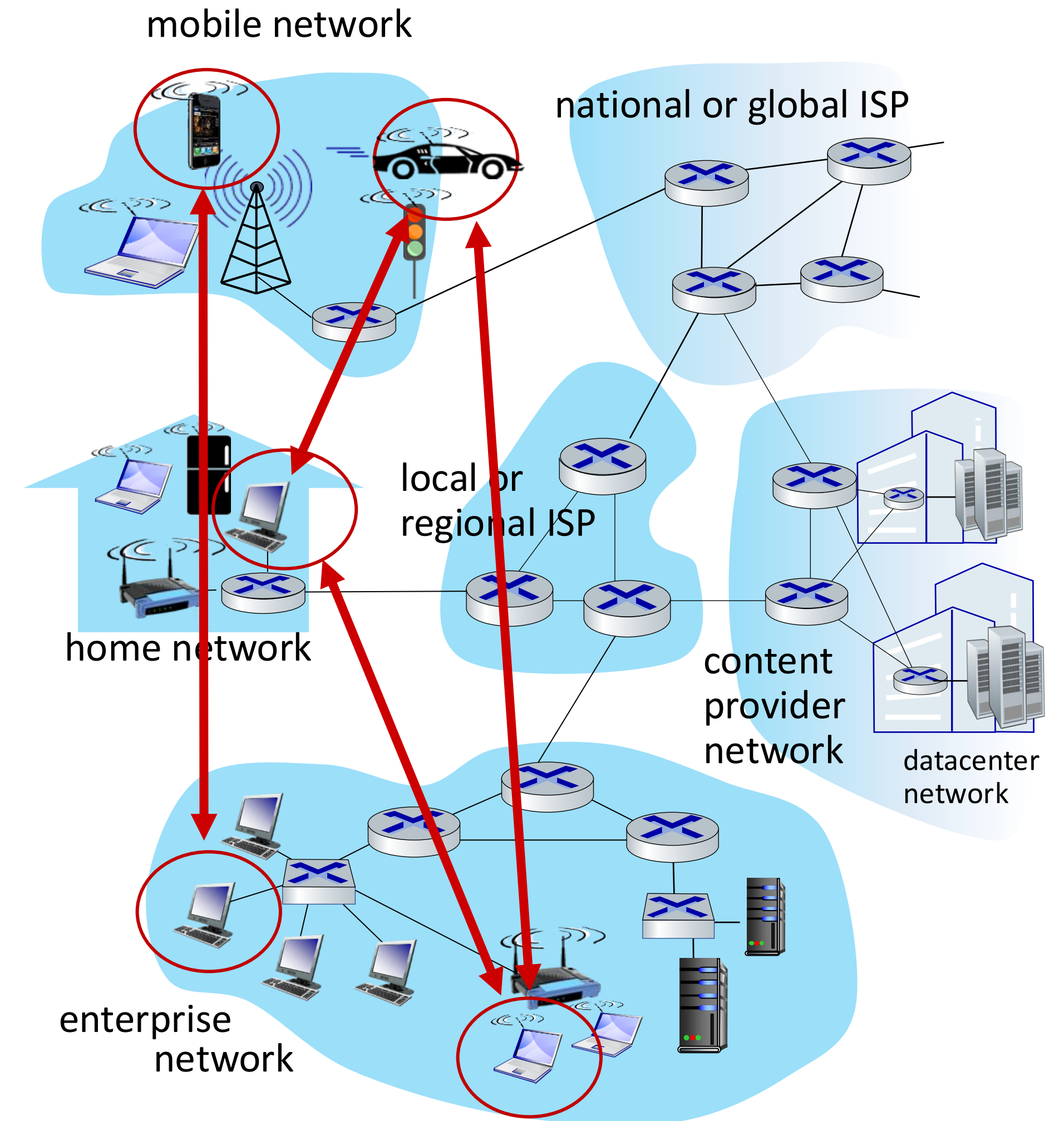
## ► Clients

- Contact, communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do not communicate directly with each other
- Examples: HTTP, IMAP, FTP



# Peer-to-peer Architecture

- ▶ No always-on server
- ▶ Arbitrary end systems directly communicate
- ▶ Peers request service from other peers, provide service in return to other peers
  - ▶ Self-scalability – New peers bring new capacity, as well as new service demands
- ▶ Peers are intermittently connected and change (IP) address
  - ▶ Complex management
- ▶ Example: P2P file sharing (BitTorrent)



Adapted from slides by J.F. Kurose and K.W. Ross

# Process Communicating

## ▶ Process

- ▶ Program running within a host
- ▶ Within same host, processes communicate inter-process communication (OS)
- ▶ Processes in different hosts communicate exchanging messages

## ▶ Clients & Servers

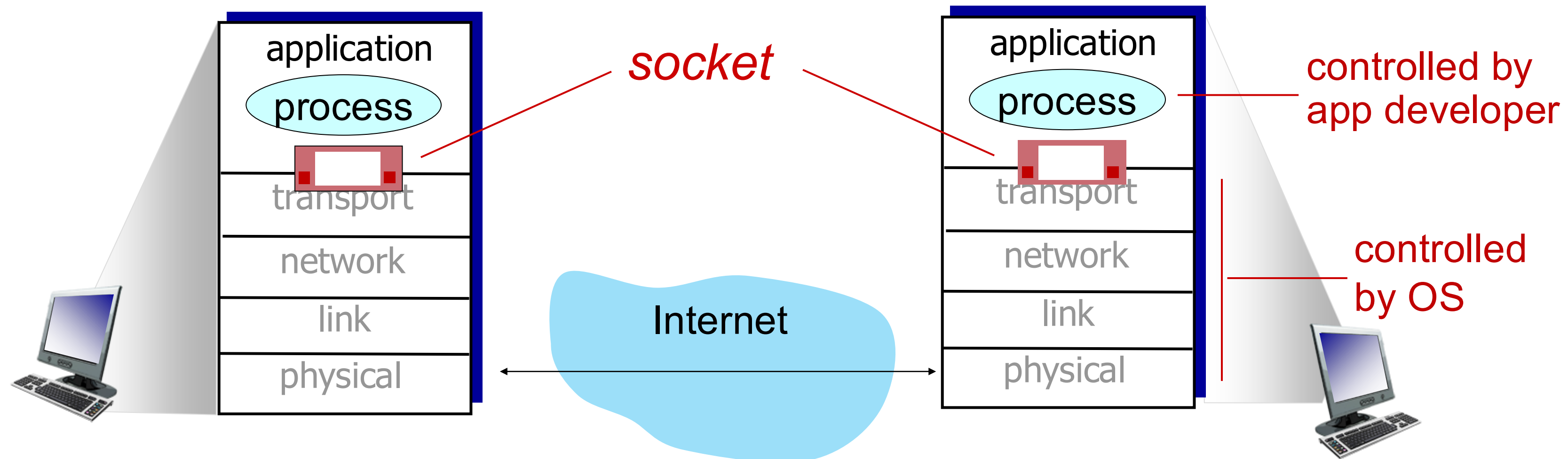
- ▶ Client processes initiate communication
- ▶ Server processes wait to be contacted

## ▶ Peer-to-peer

- ▶ P2P applications have both, client & server processes

# Sockets

- ▶ Processes send/receive messages to/from their sockets
- ▶ Socket analogous to door
  - ▶ Sending process shoves message out door
  - ▶ Sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - ▶ two sockets involved: one on each side



# Addressing Processes

- ▶ To receive messages, process must have identifier
- ▶ Host device has unique 32-bit IP address  Is this enough? No
- ▶ Identifier includes both IP address and port numbers associated with process on host
- ▶ Example port numbers
  - ▶ HTTP server: 80
  - ▶ Mail server: 25
- ▶ To send HTTP message to sdu.dk web server:
  - ▶ IP address: 20.105.224.27
  - ▶ port number: 80
- ▶ More shortly...

Learning Goal 129676

# What are properties of application layer protocols?

# An application layer protocol defines

- ▶ **Types of messages exchanged:** requests, responses
- ▶ **Message syntax:** what fields in messages & how fields are delineated
- ▶ **Message semantics:** meaning of information in fields
- ▶ **Rules:** when and how processes send & respond to messages
- ▶ **Open protocols:** defined in RFCs, everyone has access to protocol definition, allows for interoperability, e.g., HTTP, SMTP
- ▶ **Proprietary protocols:** e.g., Skype, Zoom

# What does an application layer protocol need to provide

## ▶ Data integrity

- ▶ Some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- ▶ Other apps (e.g., audio) can tolerate some loss

## ▶ Timing

- ▶ Some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

## ▶ Throughput

- ▶ Some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- ▶ Other apps (“elastic apps”) make use of whatever throughput they get

## ▶ Security

- ▶ Encryption, data integrity, ...

# Requirements of common applications

| Application           | Data loss     | Throughput                                 | Latency       |
|-----------------------|---------------|--------------------------------------------|---------------|
| File transfer         | No loss       | Elastic                                    | Not important |
| E-Mail                | No loss       | Elastic                                    | Not important |
| Website               | No loss       | Elastic                                    | Not important |
| Real-time audio/video | Loss-tolerant | Audio: 50kbps-1Mbps<br>Video: 10Kbps-5Mbps | 10's msecs    |
| Streaming audio/video | Loss-tolerant | Audio: 50kbps-1Mbps<br>Video: 10Kbps-5Mbps | Few secs      |
| Interactive games     | Loss-tolerant | Kbps+                                      | 10's msec     |

# Interlude: Transport Layer Protocols

## ▶ TCP

- ▶ Transmission Control Protocol
- ▶ Provides
  - ▶ Reliable transport
  - ▶ Flow control
  - ▶ Congestion control
  - ▶ Connection-oriented
- ▶ Does not provide
  - ▶ Timing
  - ▶ QoS guarantees
  - ▶ Security

## ▶ UDP

- ▶ User Datagram Protocol
- ▶ Provides
  - ▶ Unreliable transport
- ▶ Does not provide
  - ▶ Timing
  - ▶ QoS guarantees
  - ▶ Security
  - ▶ Flow control
  - ▶ Congestion control

**We will go into detail on these two in the lecture on the Transport Layer**

Learning Goal A530AC

# What is HTTP?

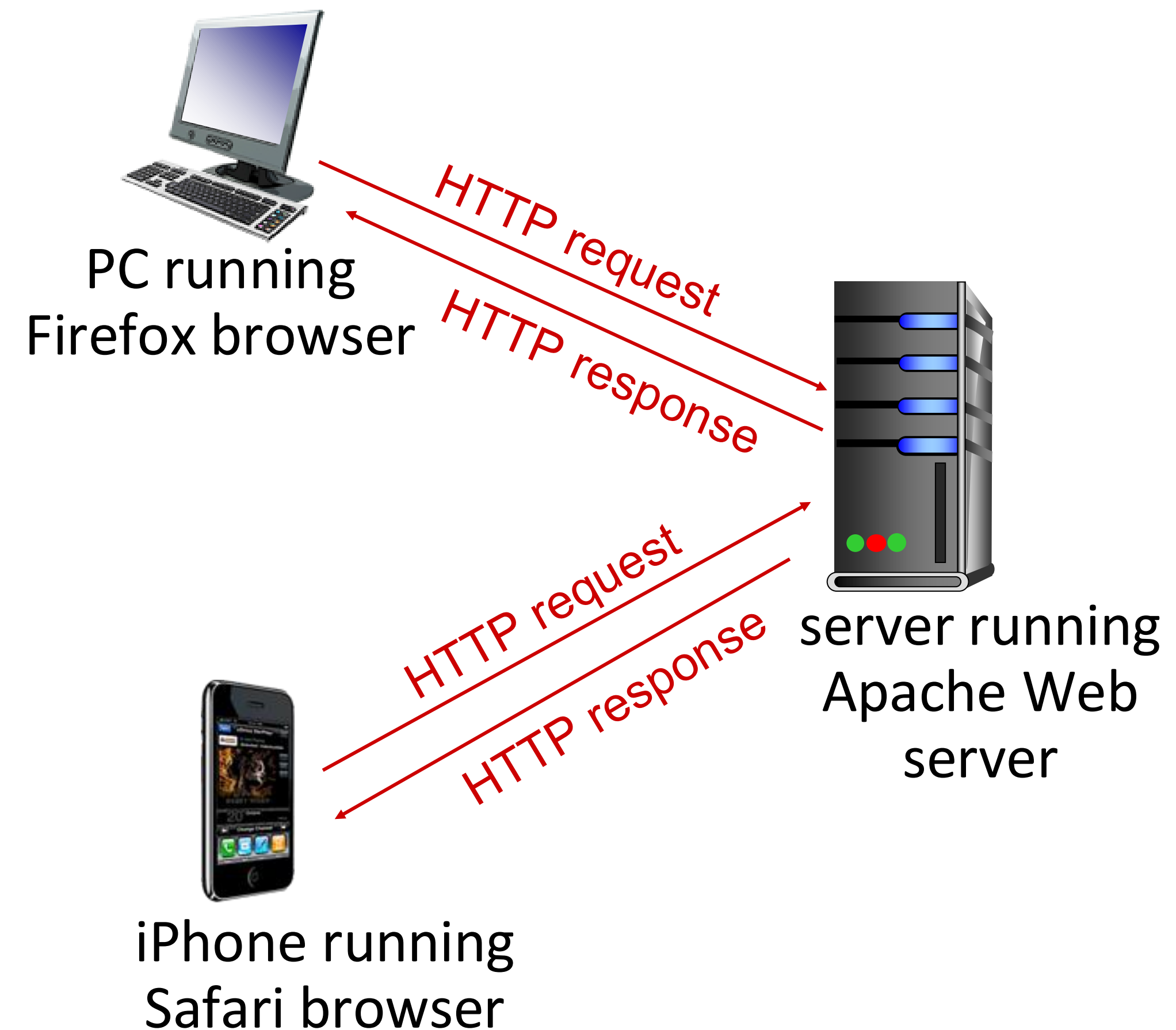
# HTTP – Hypertext Transfer Protocol

- ▶ The Internet's application layer protocol

- ▶ Client-Server Architecture

- ▶ **Client:** Browser that requests, receives, & displays websites

- ▶ **Server:** Web server sends objects of website (images, HTML files, etc.) in response to requests



# HTTP – Hypertext Transfer Protocol

- ▶ HTTP uses TCP
  - ▶ Client initiates connection (creates socket) to port 80 of the server
  - ▶ Server accepts TCP connection from client
  - ▶ HTTP messages exchanged between browser (HTTP client) and web server (HTTP server)
  - ▶ TCP connection closed
- ▶ HTTP is “stateless”
  - ▶ Server maintains no information about past client requests

## Interlude

- ”Stateful” protocols are complex
- ▶ Past history (state) must be maintained
  - ▶ In case of interruption at any of the sides, state might become inconsistent and must be reconciled

# HTTP – Non-Persistence vs Persistence

## ► Non-Persistent HTTP (HTTP/1.0)

1. TCP Connection opened
2. At most one object sent over TCP connection  
---> Downloading multiple objects requires multiple connections
3. TCP connection closed

## ► Persistent

1. TCP connection opened
2. Multiple objects can be sent over single TCP connection
3. TCP connection closed

# Non-Persistent HTTP – Procedure

User enters URL:

[www.sdu.dk/en/om\\_sdu/institutter\\_centre/imada\\_matematik\\_og\\_dataologi/index.html](http://www.sdu.dk/en/om_sdu/institutter_centre/imada_matematik_og_dataologi/index.html)

Suppose the site contains text and references to 10 images



**1a.** HTTP client initiates TCP connection to HTTP server (process) at `www.sdu.dk` on port 80



**1b.** HTTP server at host `www.sdu.dk` waiting for TCP connection at port 80 “accepts” connection, notifying client

time

**2.** HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object `en/om_sdu/.../index.html`

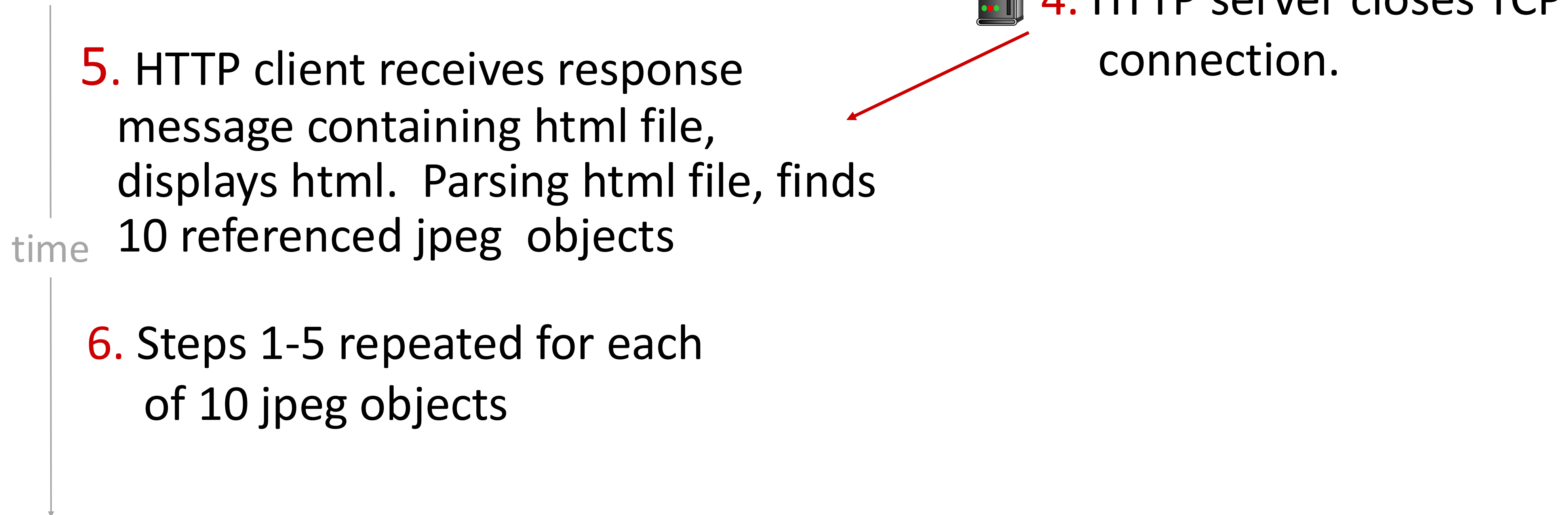
**3.** HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

# Non-Persistent HTTP – Procedure

User enters URL:

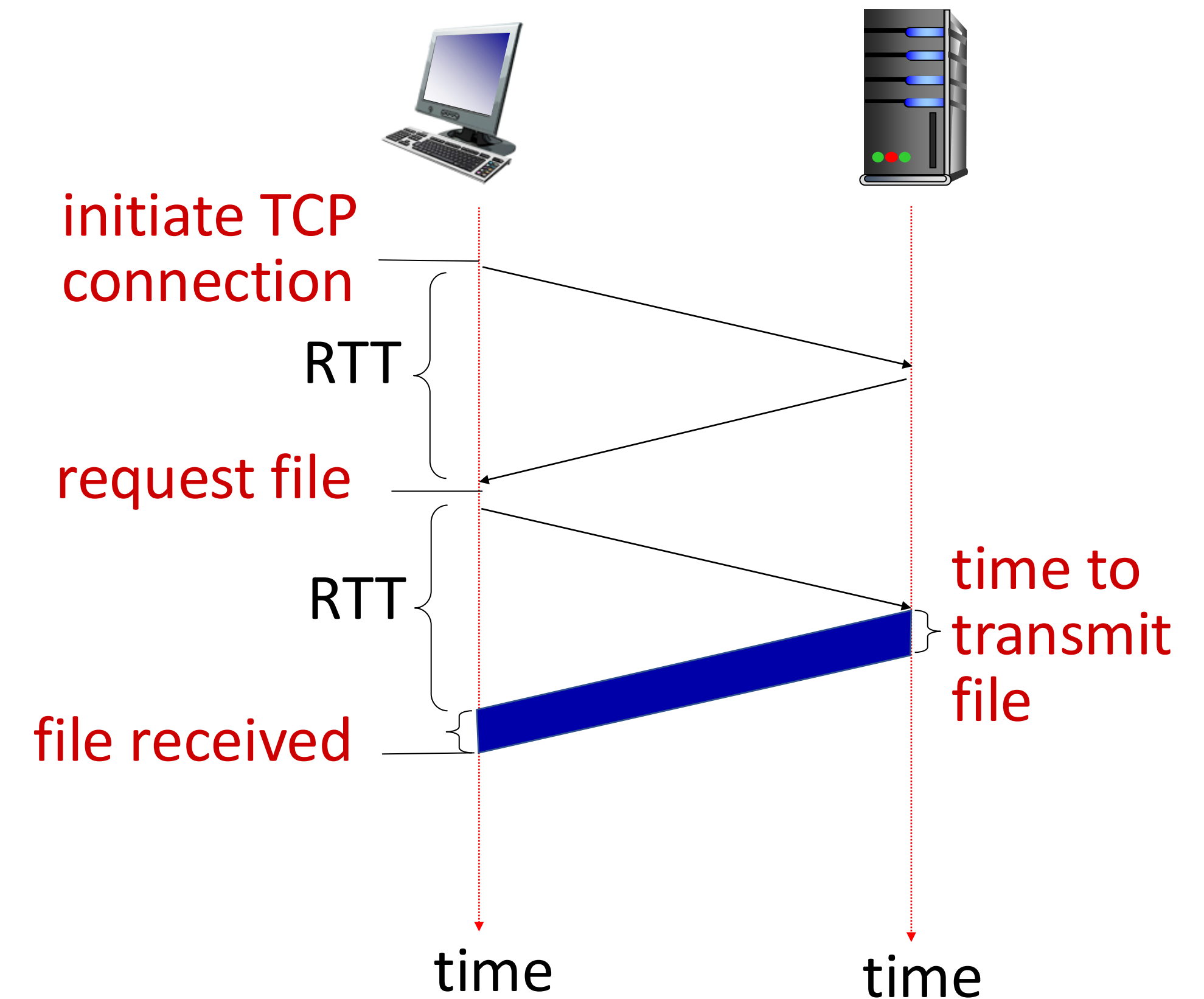
[www.sdu.dk/en/om\\_sdu/institutter\\_centre/imada\\_matematik\\_og\\_dataologi/index.html](http://www.sdu.dk/en/om_sdu/institutter_centre/imada_matematik_og_dataologi/index.html)

Suppose the site contains text and references to 10 images



# Non-Persistent HTTP – Response Time

- ▶ **Round-Trip Time (RTT):** Time for a small packet to travel from client to server and back
- ▶ HTTP response time
  - ▶ One RTT to initiate TCP connection
  - ▶ One RTT for HTTP request and first few bytes of HTTP response to return
  - ▶ Object/file transmission time



- ▶ Non-persistent HTTP response time = 2 RTT + file transmission time

# Persistent HTTP (HTTP/1.1)

- ▶ Issues with non-persistent HTTP
  - ▶ Requires 2 RTT per object
  - ▶ OS overhead for each TCP connection
  - ▶ Browsers often open multiple parallel TCP connections to fetch in parallel
- ▶ Solution: Persistent HTTP
  - ▶ Server leaves connection open after sending response
  - ▶ Subsequent HTTP messages sent over open connection
  - ▶ Client sends requests as soon as it encounters a referenced object
  - ▶ As little as one RTT for all the referenced objects

# HTTP Requests

- ▶ Two types of HTTP messages: request, response
- ▶ HTTP request message: ASCII (human-readable format)

request line (GET, POST, HEAD commands)

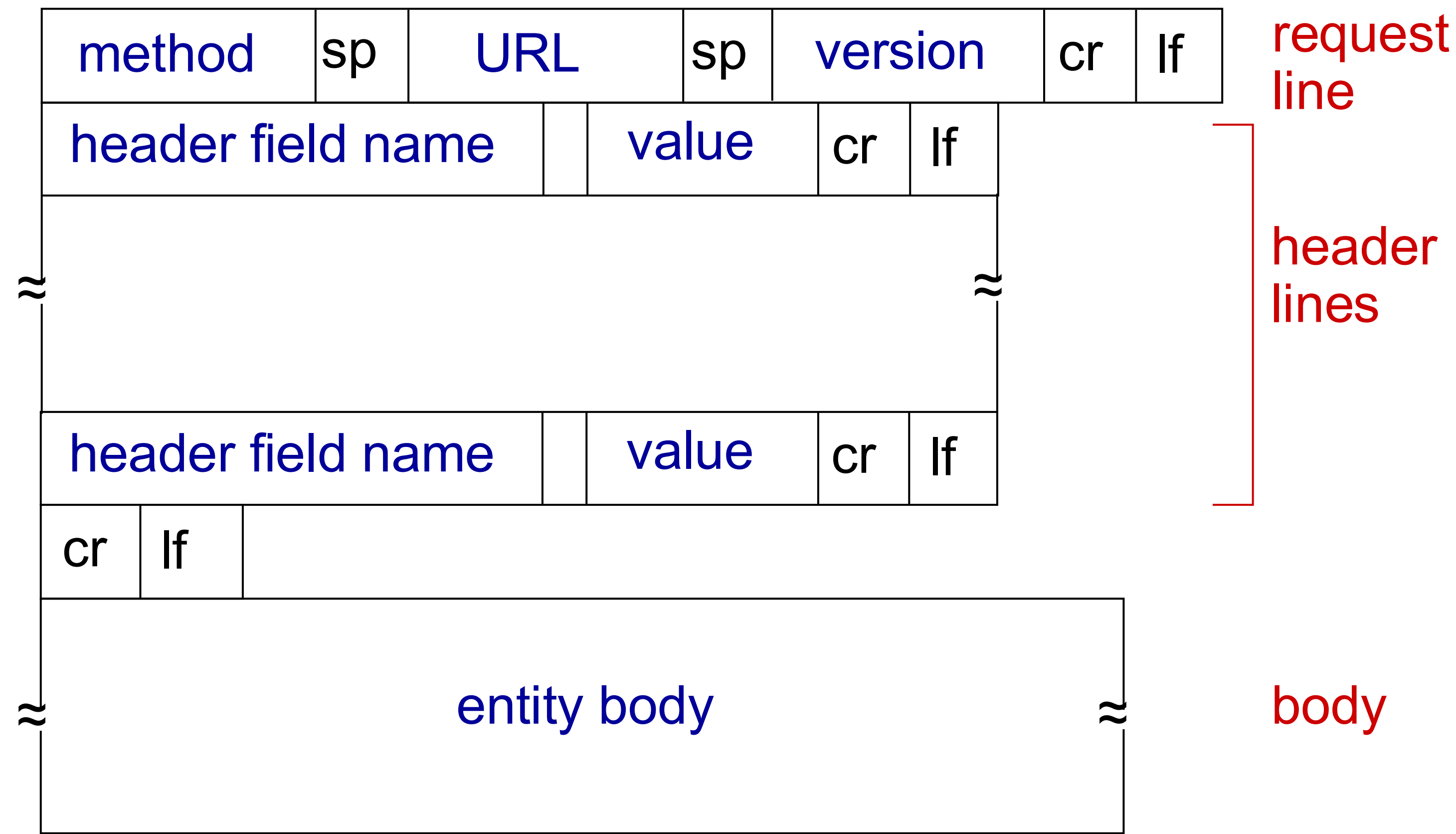
header lines

carriage return character  
line-feed character

```
GET /index.html HTTP/1.1\r\n
Host: www.sdu.dk\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
rv:109.0) Gecko/20100101 Firefox/116.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return, line feed  
at start of line indicates  
end of header lines

# HTTP Request Format



# Important HTTP Requests

- ▶ **GET:** Sends data to server in URL field by using '?' and '&':

`www.sdu.dk?monkeywants=banana&...&parameter_n=value_n`

- ▶ **POST:** Sends data to server in message body, e.g. form input
- ▶ **HEAD:** Requests headers (only) that would be returned if specified URL were requested with an HTTP GET method
- ▶ **PUT:** Uploads new file (object) to server in message body completely replacing existing object at specified URL

# HTTP Responses

status line (protocol  
status code status phrase)

header  
lines

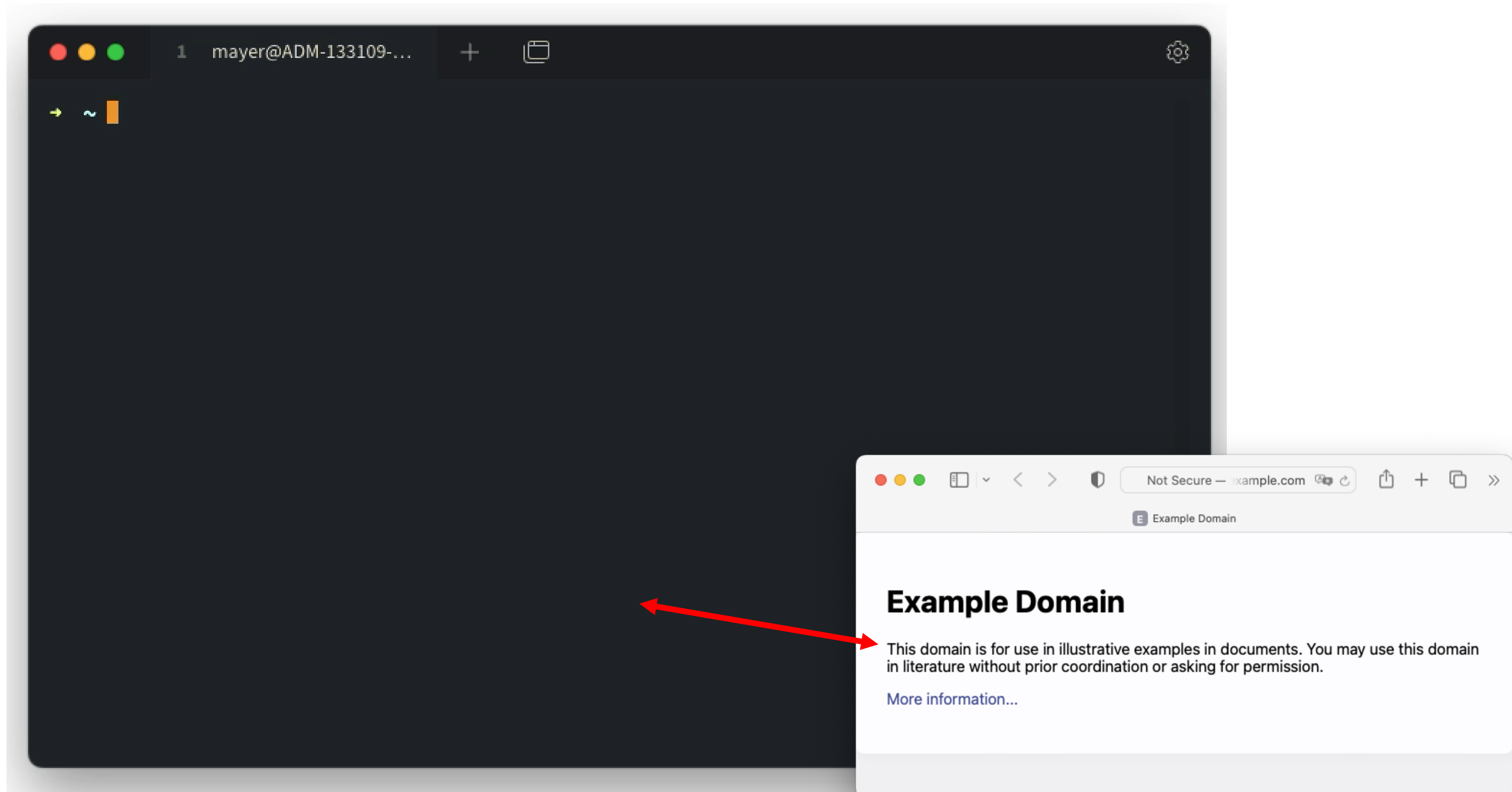
data, e.g., requested  
HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-
      fips PHP/7.4.9 mod_perl/2.0.11
      Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

# HTTP Response Status Codes

- ▶ Status code appears in 1st line in server-to-client response message
- ▶ Some sample codes
  - ▶ **200 OK:** request succeeded, requested object later in this message
  - ▶ **301 Moved Permanently:** requested object moved, new location specified later in this message (in Location: field) → Redirect saved in clients
  - ▶ **302 Moved Temporarily:** requested object moved, new location specified later in this message (in Location: field) → Redirect saved in clients
  - ▶ **400 Bad Request:** Request message not understood by server
  - ▶ **403 Forbidden:** Request is understood, but client is not authorised
  - ▶ **404 Not found:** Requested document not found on this server
  - ▶ **418 ???:** Homework to find out yourself

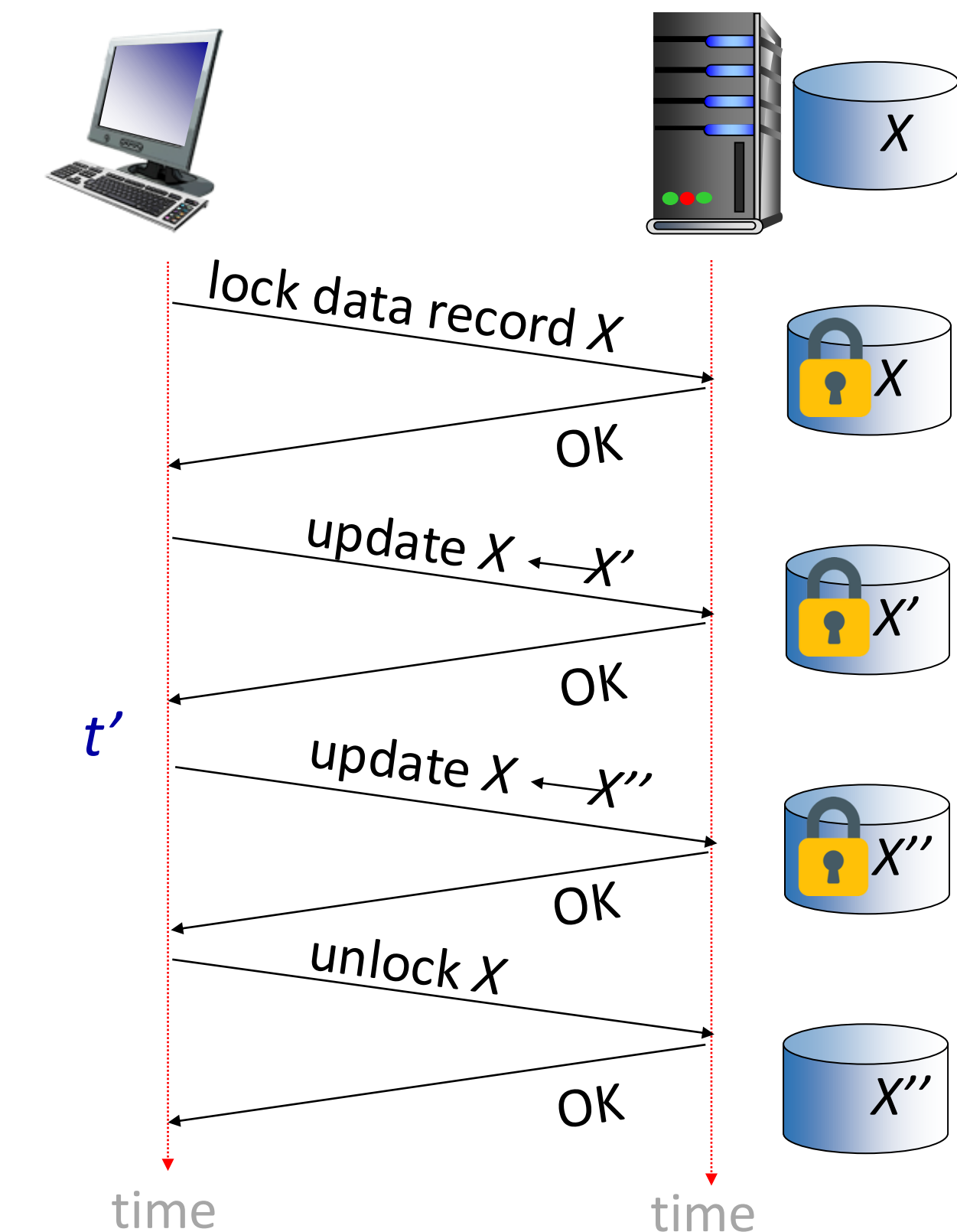
# HTTP in action



# Maintaining State – A stateful protocol?

- ▶ Recall: HTTP is stateless
- ▶ No notion of multi-step exchanges of HTTP messages to complete a “transaction”
- ▶ No need for client/server to track “state” of multi-step exchange
- ▶ All HTTP requests are independent of each other
- ▶ No need for client/server to “recover” from a *partially-completed-but-never-completely-completed* transaction

**Stateful protocol:** client makes two changes to  $X$ , or none at all



**Q:** what happens if network connection or client crashes at  $t'$ ?

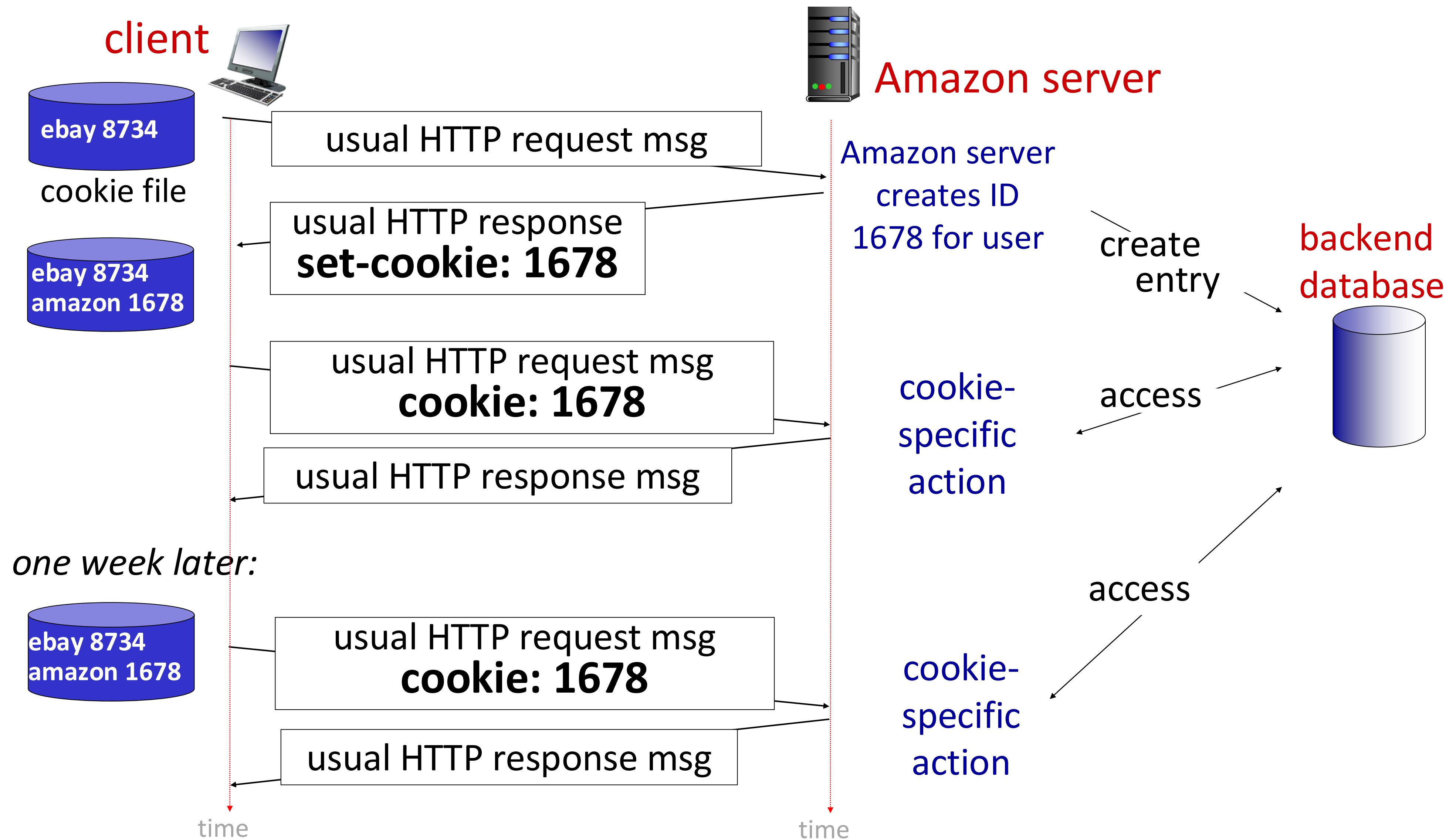
# Maintaining State – Cookies

- ▶ Websites and client browser use cookies to maintain some state between transactions
- ▶ Four components:
  1. Cookie header line of HTTP response message
  2. Cookie header line in next HTTP request message
  3. Cookie file kept on user's host, managed by user's browser
  4. Back-end database at Website

## Example

- ▶ Susan uses browser on laptop to visit specific e-commerce site for first time
- ▶ When initial HTTP requests arrive at site, site creates
  - ▶ Unique ID
  - ▶ Entry in backend DB for ID
- ▶ Subsequent HTTP requests from Susan to this site will contain cookie with ID value, allowing site to “identify” Susan

# Maintaining State – Cookies



# Cookies – Some further notes

- ▶ What cookies can be used for:
  - ▶ Authorization
  - ▶ Shopping carts
  - ▶ Recommendations
  - ▶ User session state (Web e-mail)
- ▶ Challenge: How to keep state?
  - ▶ At clients (protocol endpoints): maintain state at sender/receiver over multiple transactions
  - ▶ In messages: cookies sent in HTTP messages

## Note

### Cookies and Privacy

- ▶ Cookies permit sites to learn a lot about you on their site
- ▶ 3<sup>rd</sup>-party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

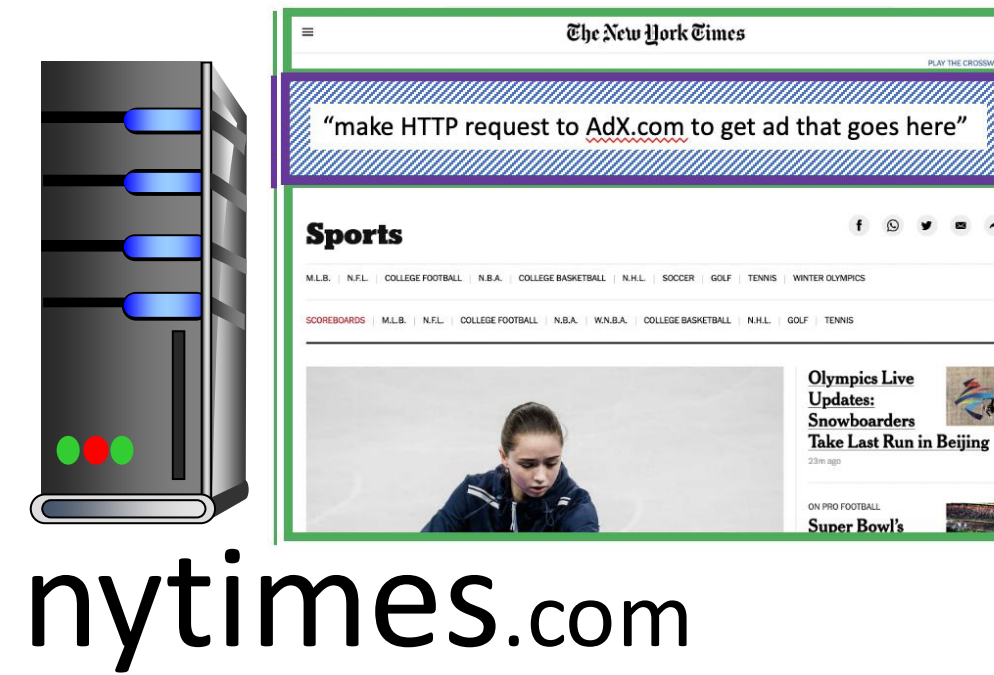
## Note

### Cookies and Security

- ▶ Cookies can contain secrets, e.g., session IDs with payment info
- ▶ They can have the `secure` attribute which means that the cookies only wants to be sent over secure (HTTPS) connections

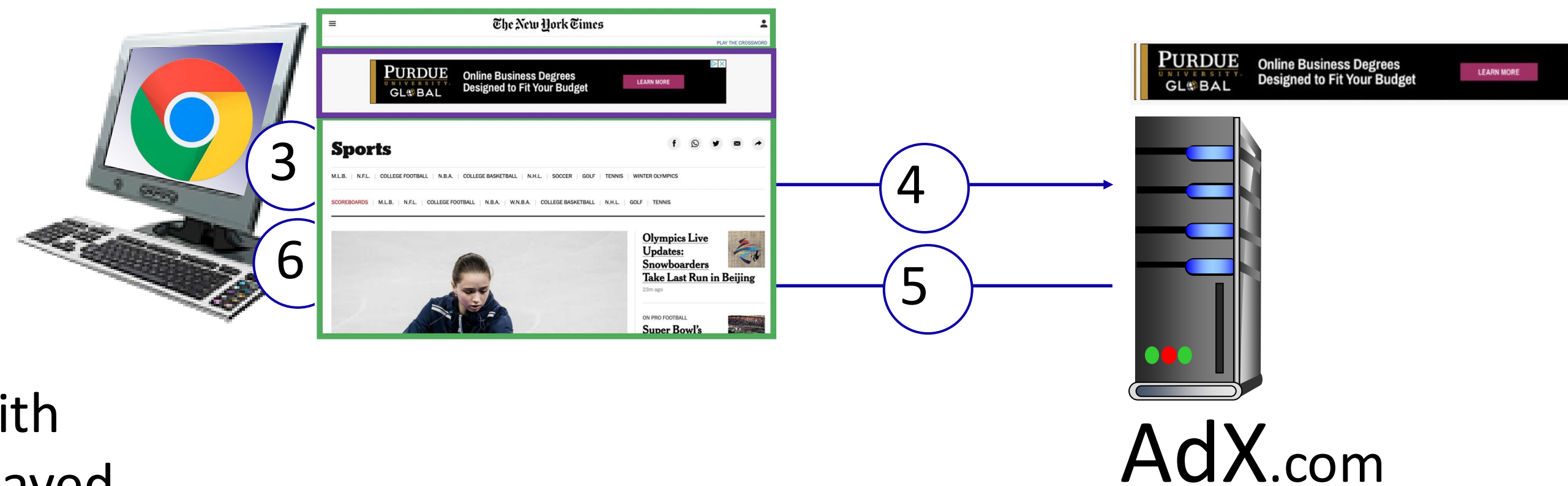
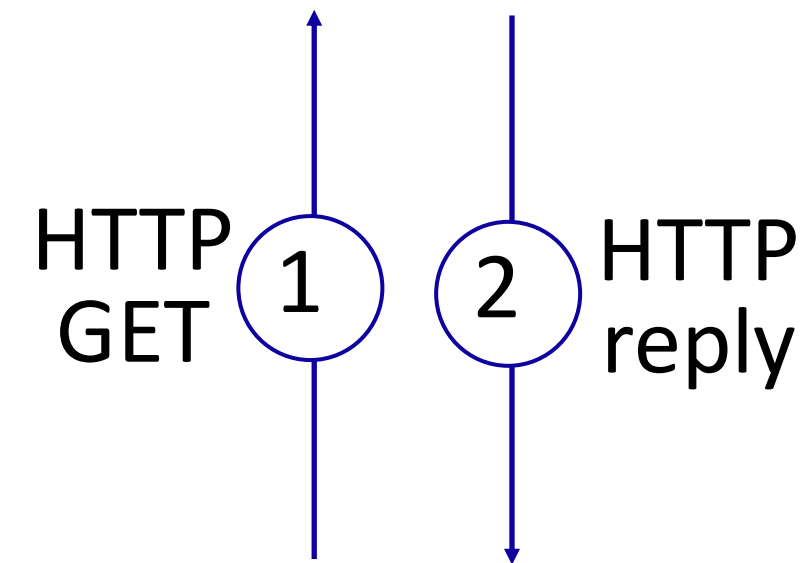
# A Cookie Example – Opening the NY Times website

1 GET base html file  
2 from nytimes.com

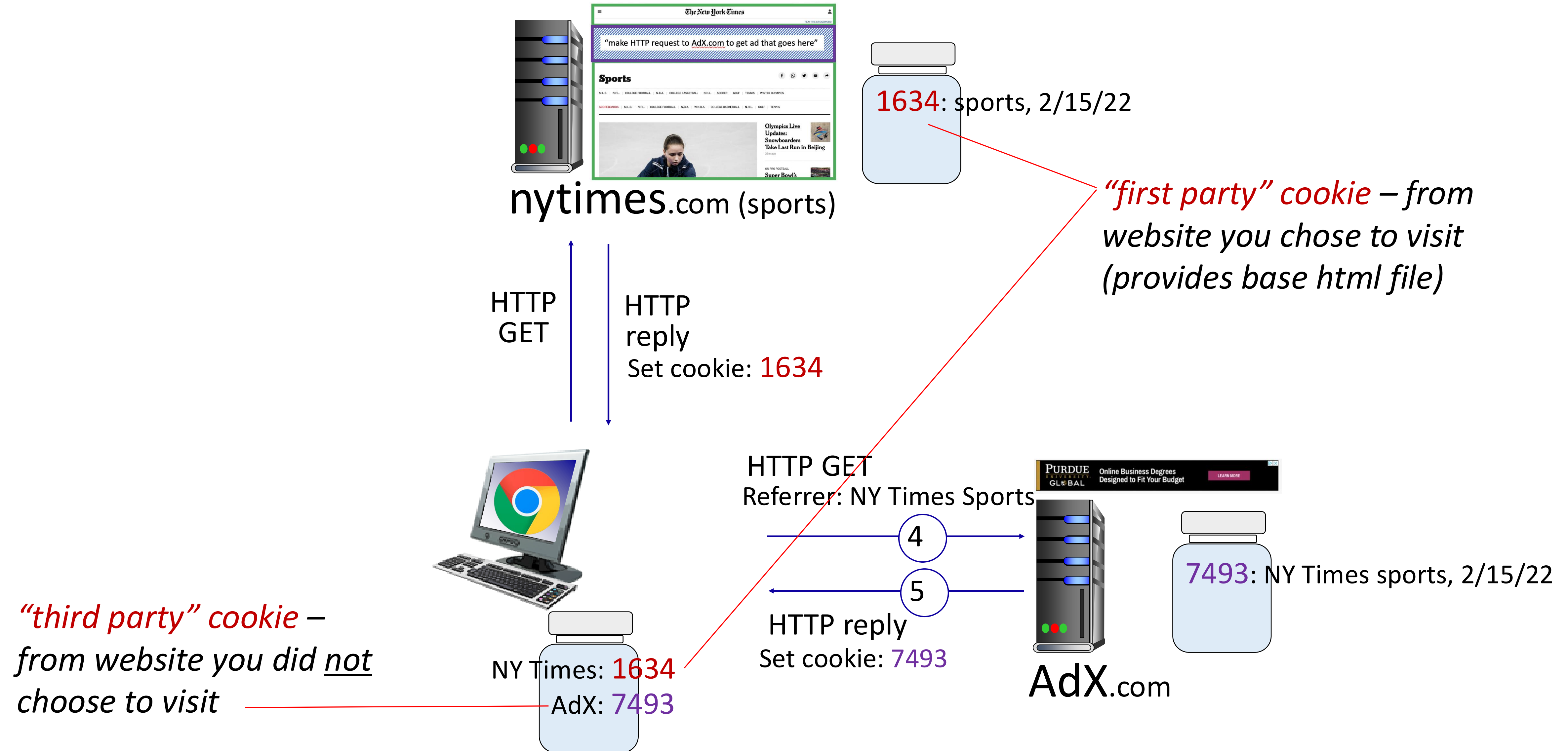


4 fetch ad from  
5 AdX.com

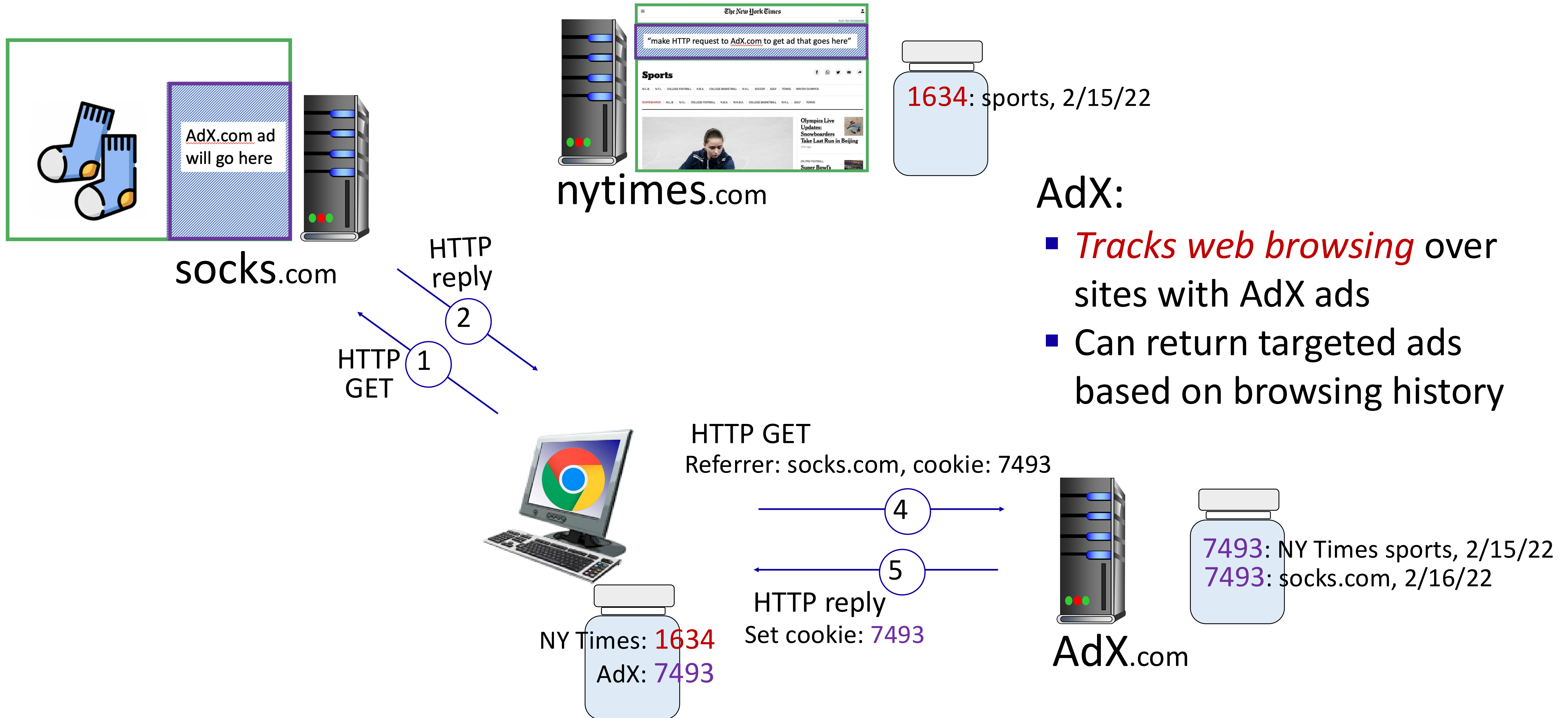
7 display composed  
page



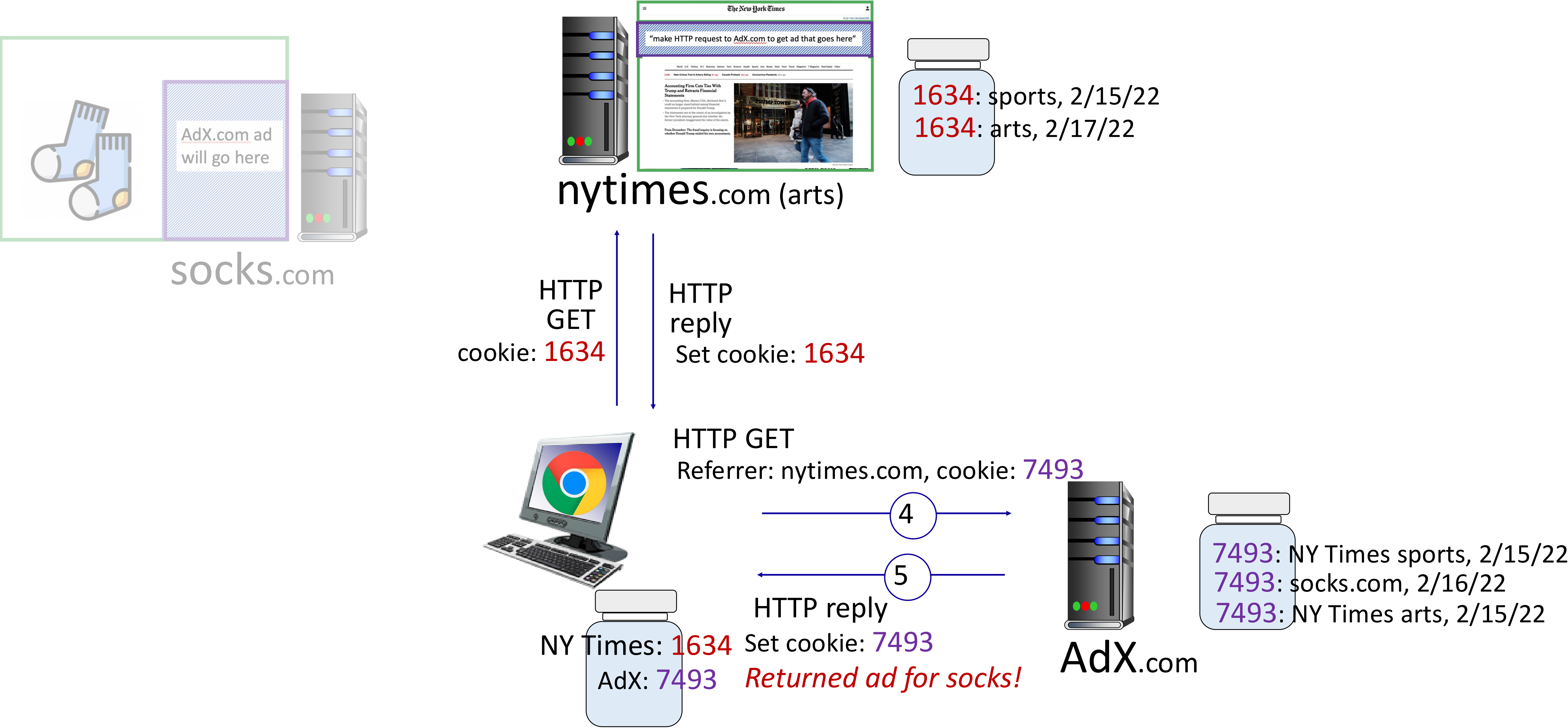
# A Cookie Example – Tracking a user's behaviour



# A Cookie Example – Tracking a user's behaviour



# A Cookie Example – Tracking a user’s behaviour



Adapted from slides by J.F. Kurose and K.W. Ross

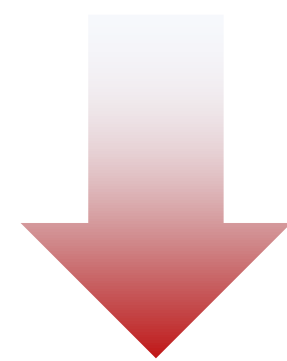
# Tracking a user's behaviour

- ▶ Cookies can be used to
  - ▶ Track user behaviours on a given website (first party cookies)
  - ▶ Track user behaviours across multiple websites (third party cookies) without user ever choosing to visit tracker site (!)
  - ▶ Tracking may be invisible to user:
    - ▶ Rather than displayed ad triggering HTTP GET to tracker, could be an invisible link
- ▶ Third party tracking via cookies
  - ▶ Disabled by default in Firefox, Safari browsers
  - ▶ To be disabled in Chrome browser in 2023

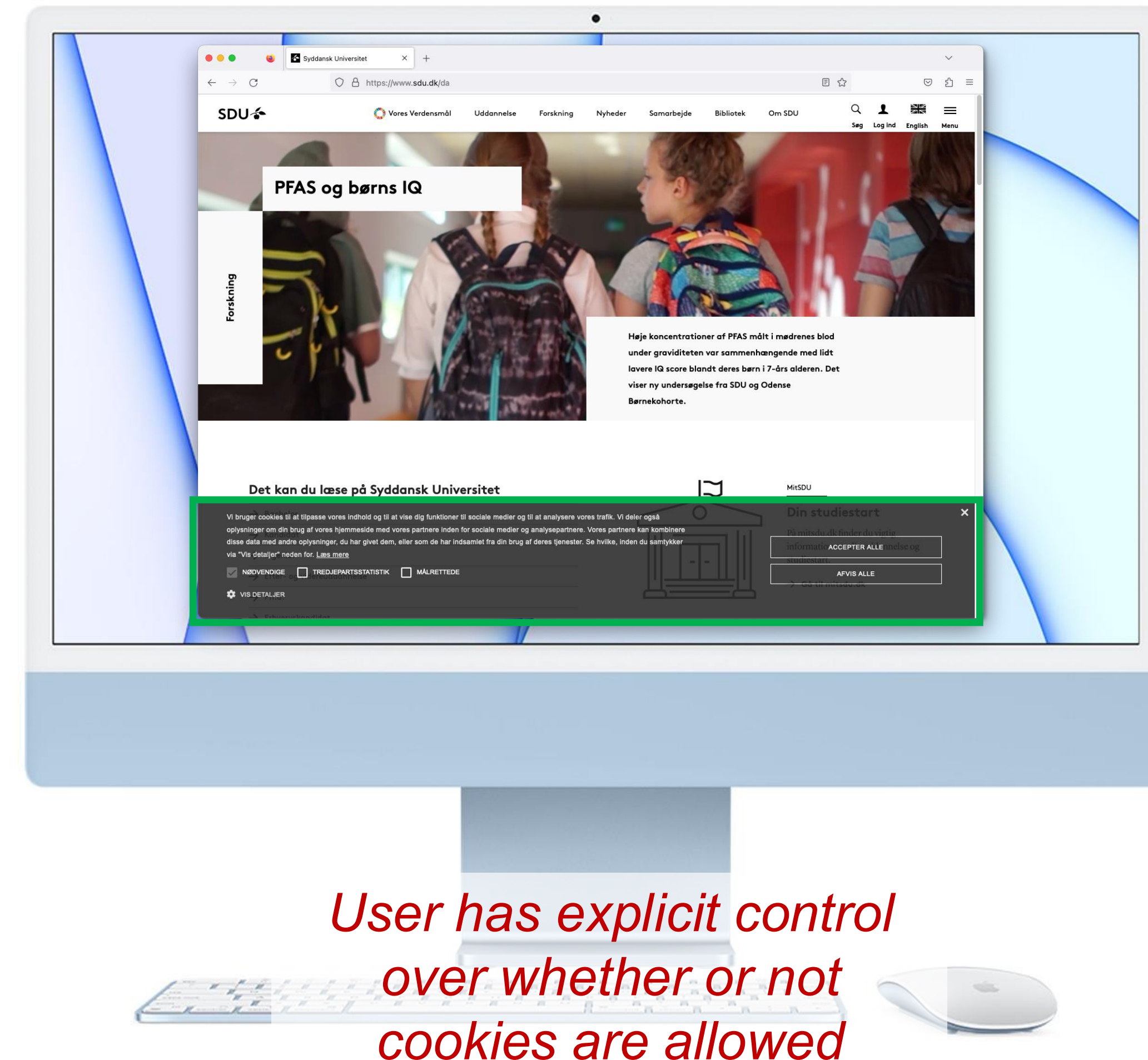
# General Data Protection Regulation (GDPR) & Cookies

“Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...]. This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”

GDPR, recital 30 (May 2018)



When cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations

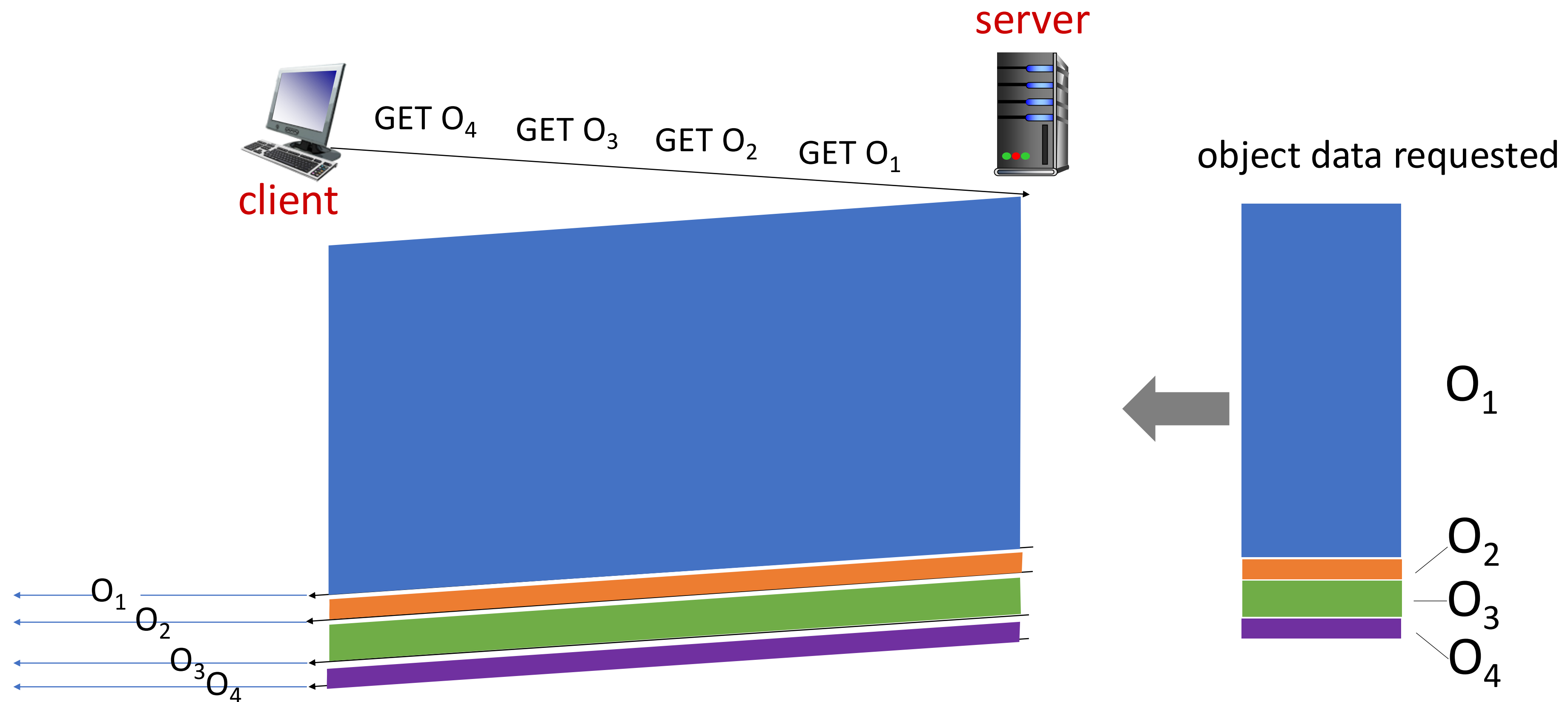


# HTTP/2

- ▶ **Key goal:** Decreased delay in multi-object HTTP requests
- ▶ HTTP Versions
  - ▶ **HTTP/1.0:** Initial HTTP version, Individual TCP connections per object
  - ▶ **HTTP/1.1:** Introduced multiple, pipelined GETs over single TCP connection
    - ▶ Server responds in-order (FCFS, first-come-first-served) to GET requests
    - ▶ With FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
    - ▶ Loss recovery (retransmitting lost TCP segments) stalls object transmission

# HOL Blocking in HTTP 1.1

HTTP 1.1: client requests 1 large object and 3 smaller objects



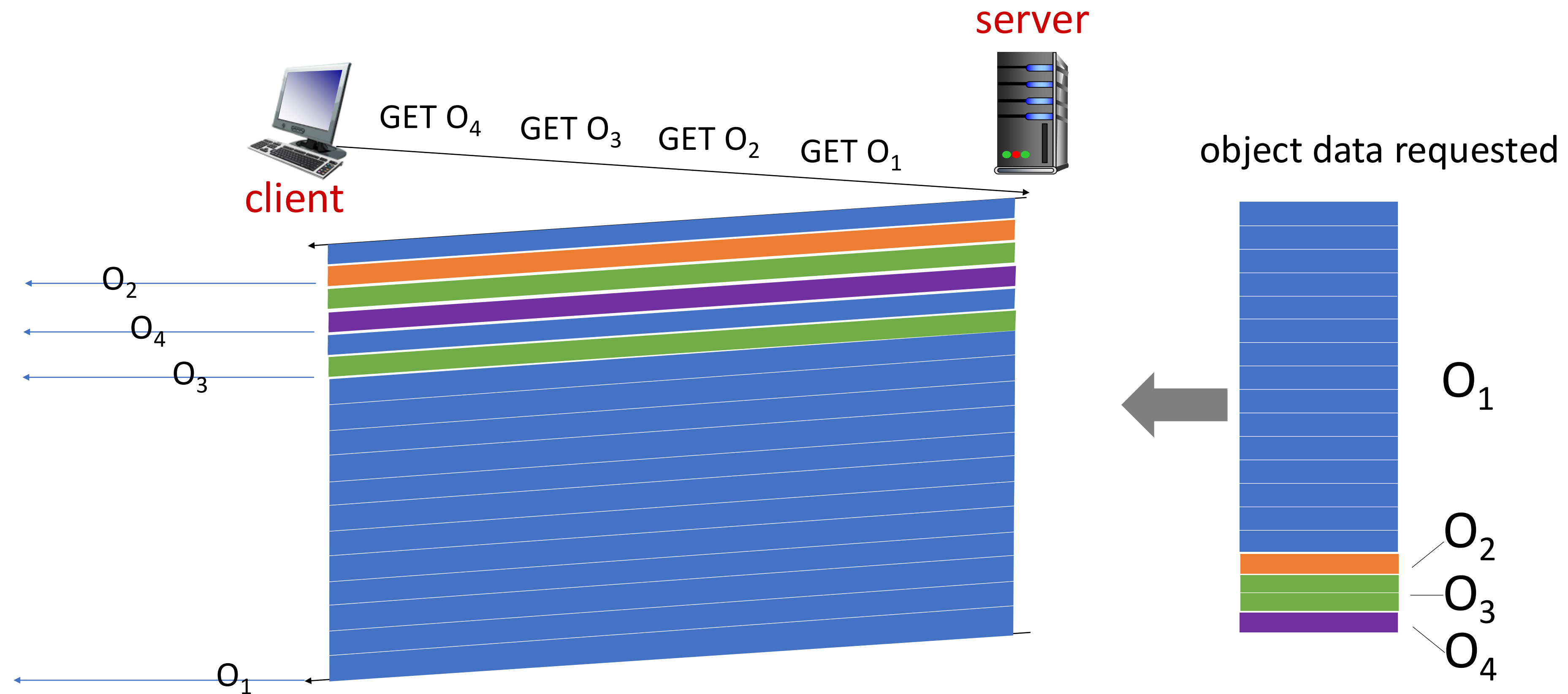
*objects delivered in order requested: O<sub>2</sub>, O<sub>3</sub>, O<sub>4</sub> wait behind O<sub>1</sub>*

# HTTP/2 to the rescue

- ▶ **Key goal:** Decreased delay in multi-object HTTP requests
- ▶ HTTP Versions
  - ▶ **HTTP/1.0:** Initial HTTP version, Individual TCP connections per object
  - ▶ **HTTP/1.1:** Introduced multiple, pipelined GETs over single TCP connection
  - ▶ **HTTP/2:** Increased flexibility at server in sending objects
    - ▶ Methods, status codes, most header fields unchanged from HTTP 1.1
    - ▶ Transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
    - ▶ Push unrequested objects to client
    - ▶ Divide objects into frames, schedule frames to mitigate HOL blocking

# HTTP/2 – Mitigating HOL Blocking

HTTP/2: objects divided into frames, frame transmission interleaved



*O<sub>2</sub>, O<sub>3</sub>, O<sub>4</sub> delivered quickly, O<sub>1</sub> slightly delayed*

# HTTP/2 to HTTP/3

- ▶ **HTTP/1.0:** Initial HTTP version, Individual TCP connections per object
- ▶ **HTTP/1.1:** Introduced multiple GETs over single TCP connection
- ▶ **HTTP/2:** Increased flexibility at server in sending objects
  - ▶ Methods, status codes, most header fields unchanged from HTTP 1.1
  - ▶ Transmission order of requested objects client-specified object priority
  - ▶ Push unrequested objects to client
  - ▶ Divide objects into frames, schedule frames to mitigate HOL blocking
  - ▶ Recovery from packet loss still stalls all object transmissions
  - ▶ As in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
  - ▶ No security over vanilla TCP connection

# HTTP/2 to HTTP/3

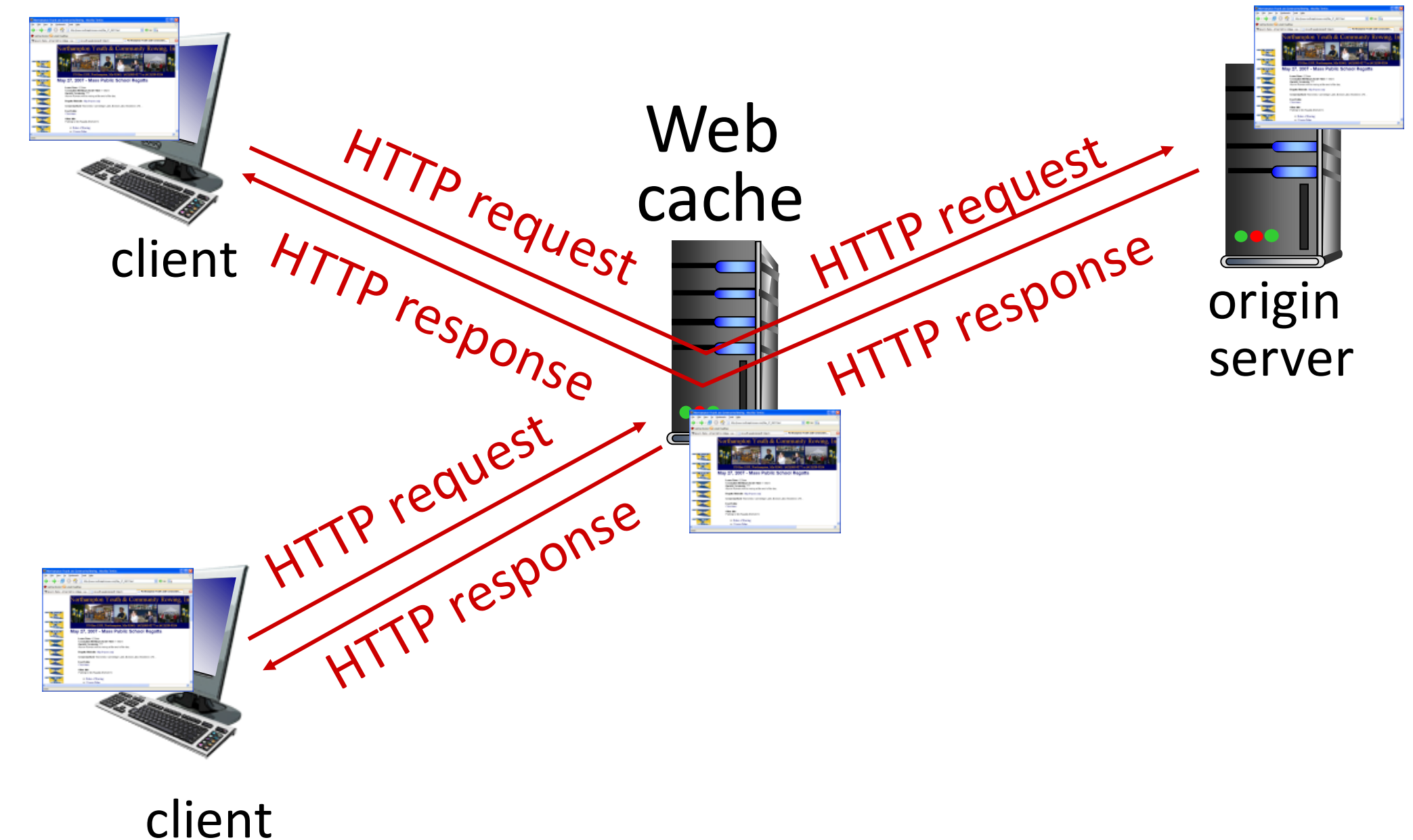
- ▶ **HTTP/1.0:** Initial HTTP version, Individual TCP connections per object
- ▶ **HTTP/1.1:** Introduced multiple GETs over single TCP connection
- ▶ **HTTP/2:** Increased flexibility at server in sending objects
- ▶ **HTTP/3:** Adds
  - ▶ Security
  - ▶ Per object error- and congestion-control (more pipelining) over UDP
  - ▶ More on HTTP/3 in transport layer lecture

Learning Goal 106AD3

# What is the role of caches in HTTP performance?

# Web Caches

- ▶ **Goal:** Satisfy client requests without involving origin server
- ▶ User configures browser to point to a (local) Web cache
  - ▶ Browser sends all HTTP requests to cache
  - ▶ If object in cache: cache returns object to client
  - ▶ Else cache requests object from origin server, caches received object, then returns object to client



# Web Caches

- ▶ Web cache acts as both client and server
  - ▶ Server for original requesting client
  - ▶ Client to origin server
- ▶ Server tells cache about object's allowable caching in response header:
  - `Cache-Control: max-age=<seconds>`
  - `Cache-Control: no-cache`
- ▶ Why Web caching?
  - ▶ Reduce response time for client request
  - ▶ Cache is closer to client
  - ▶ Reduce traffic on an institution's access link
- ▶ Internet is dense with caches
- ▶ Enables “poor” content providers to more effectively deliver content

# Browser Caching

## ► Goal

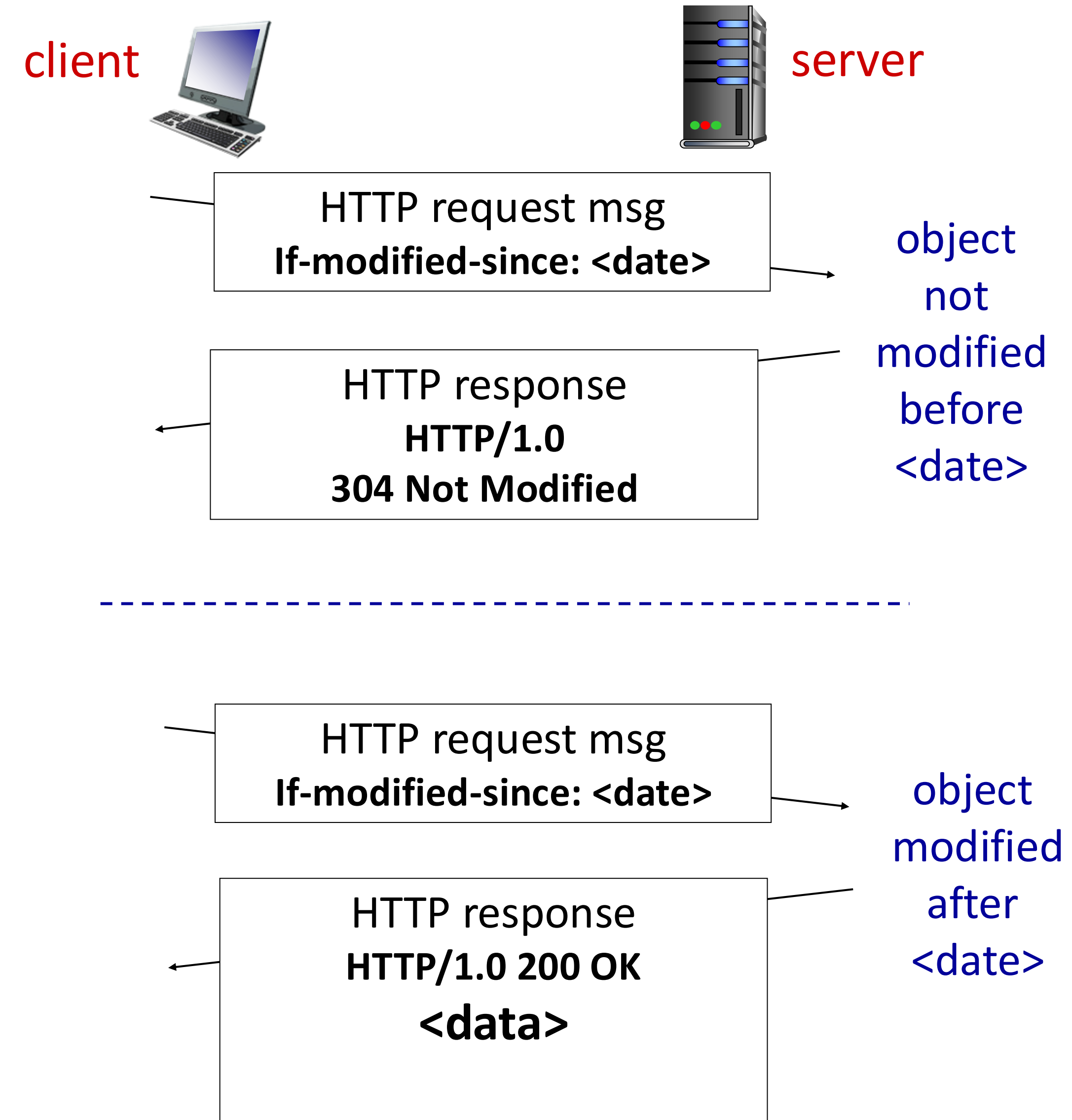
- Don't send object if browser has up-to-date cached version
- No object transmission delay (or use of network resources)

## ► Client

- Specify date of browser-cached copy in HTTP request
- `If-modified-since: <date>`

## ► Server

- response contains no object if browser-cached copy is up-to-date:
- `HTTP/1.0 304 Not Modified`

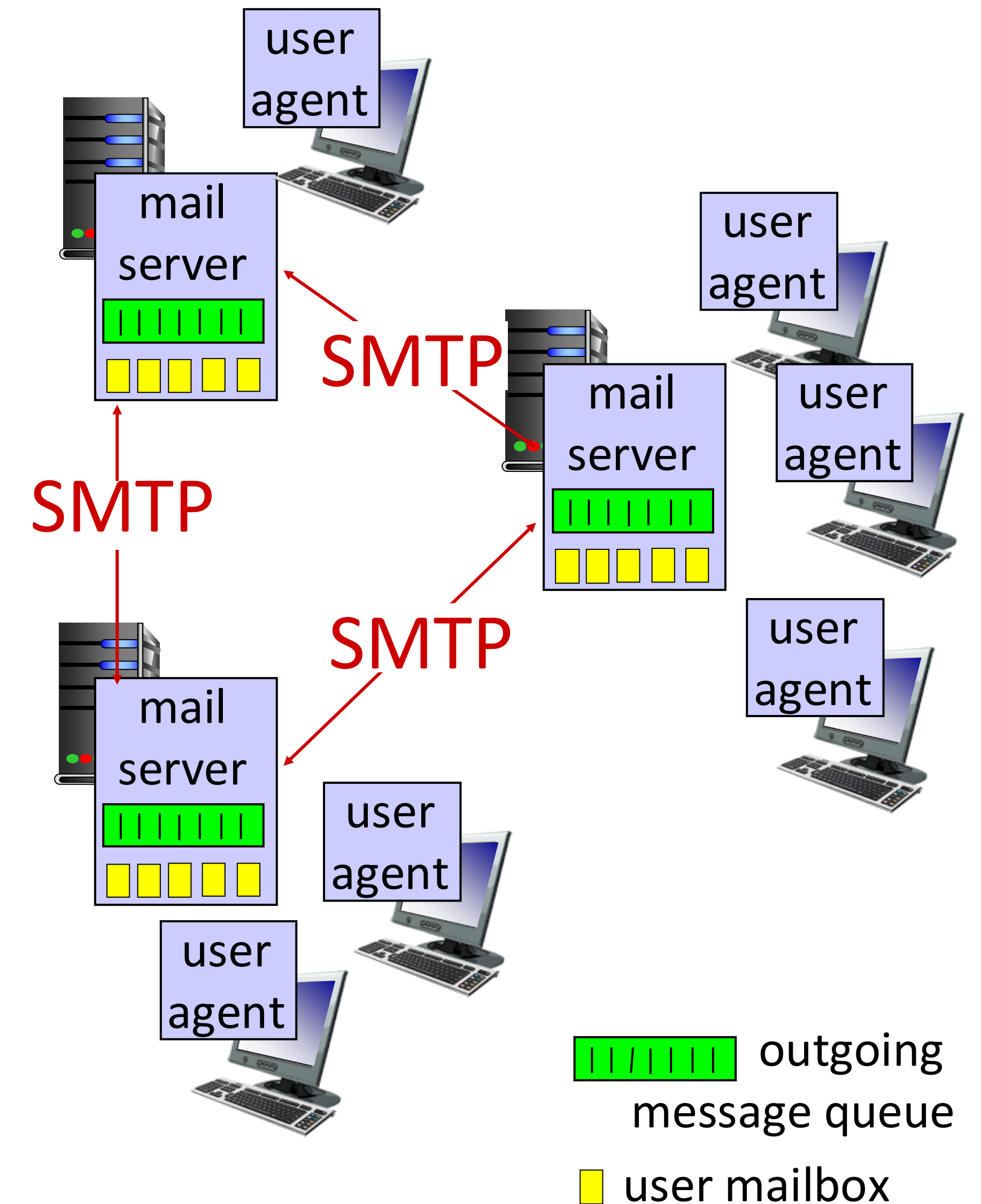


Learning Goal 642B09

**What are the protocols relevant for email?**

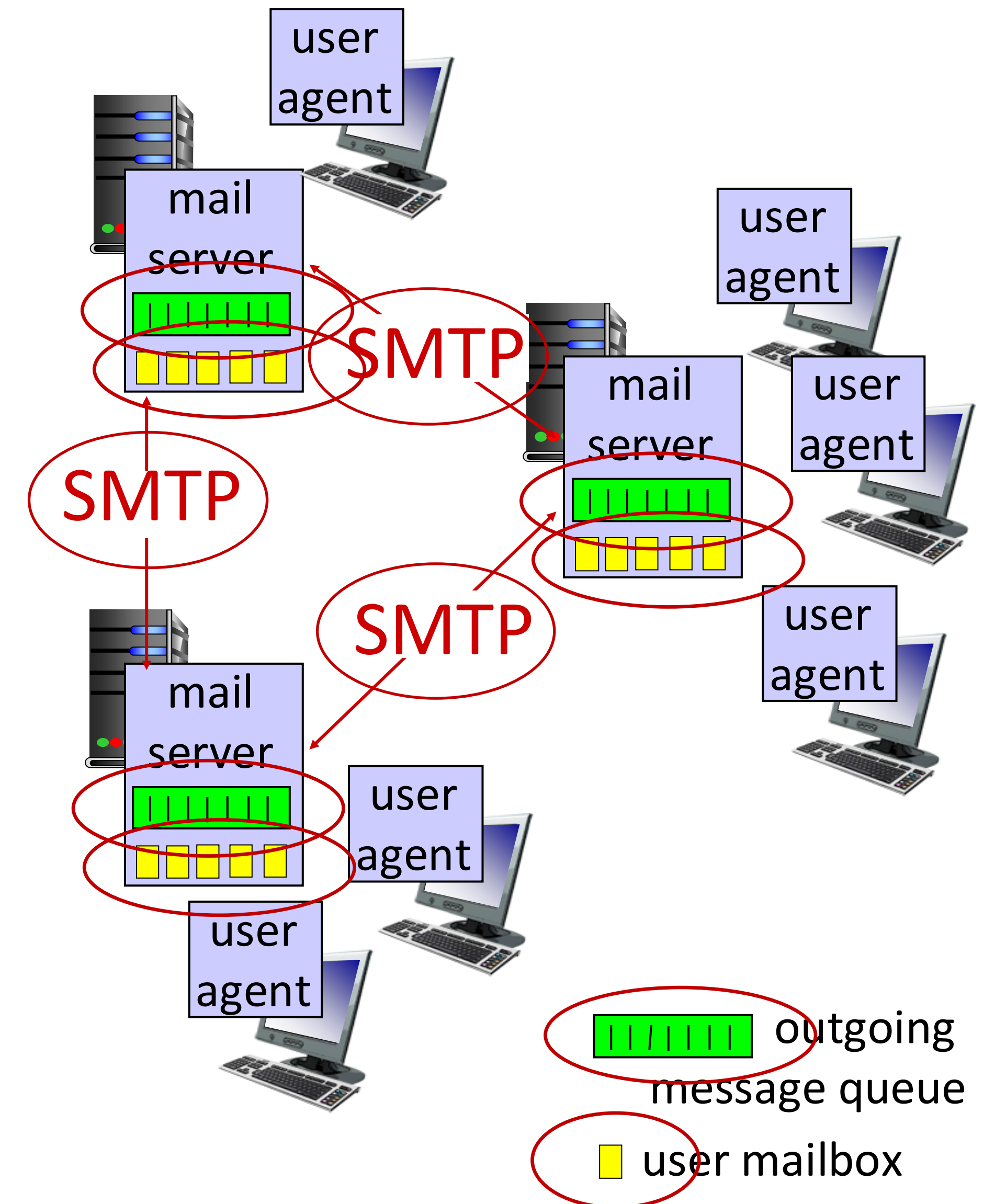
# E-mail

- ▶ Three major components
  - ▶ User agents
  - ▶ Mail servers
  - ▶ Simple mail transfer protocol: SMTP
- ▶ User Agent
  - ▶ A.k.a. “mail client”
  - ▶ Composing, editing, reading mail messages
  - ▶ E.g., Outlook, Apple Mail, Thunderbird, mutt, ...
  - ▶ Outgoing, incoming messages stored on server and fetched by client



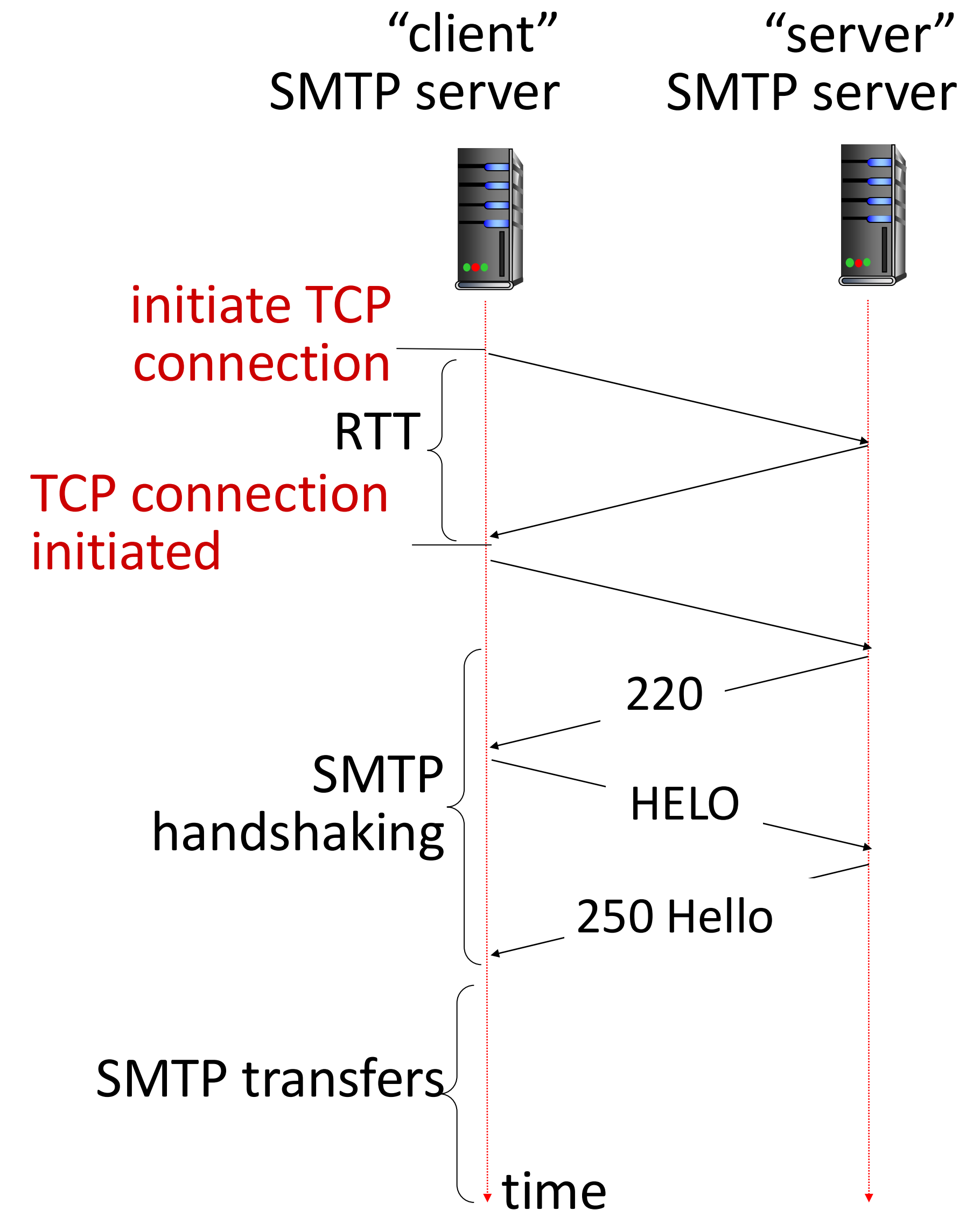
# E-mail Servers

- ▶ Mail servers
  - ▶ **Mailbox:** contains incoming messages
  - ▶ **Message queue:** outgoing (to be sent) mail messages
- ▶ SMTP protocol between mail servers to send email messages
  - ▶ **Client:** sending mail server
  - ▶ **“Server”:** receiving mail server



# Simple Mail Transfer Protocol (SMTP, RFC 5321)

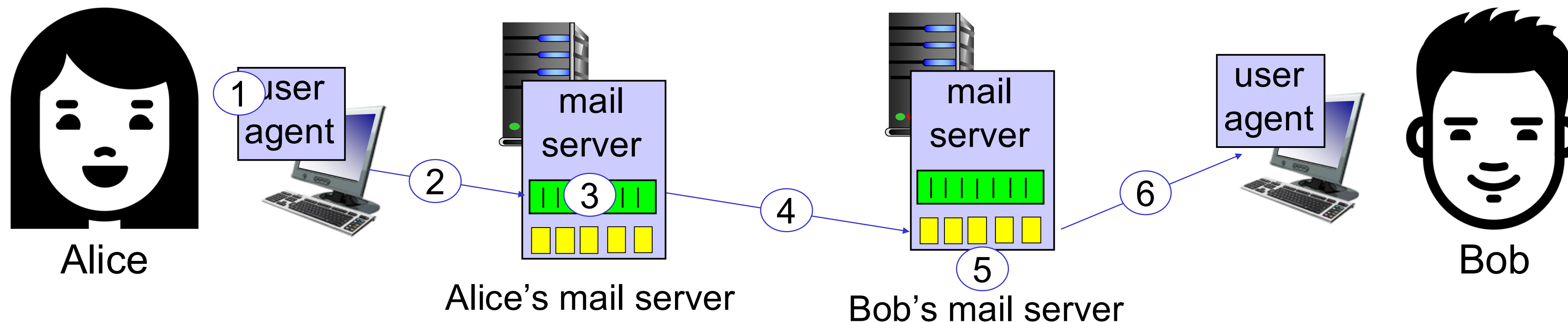
- ▶ Uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
  - ▶ Direct transfer: sending server (acting like client) to receiving server
- ▶ Three phases of transfer
  - ▶ SMTP handshaking (greeting)
  - ▶ SMTP transfer of messages
  - ▶ SMTP closure
- ▶ Command/response interaction (like HTTP)
  - ▶ Commands: ASCII text
  - ▶ Response: status code and phrase



Adapted from slides by J.F. Kurose and K.W. Ross

# Alice sends an e-mail to Bob

1. Alice uses UA to compose e-mail message “to” bob@sdu.dk
2. Alice’s UA sends message to her mail server using SMTP; message placed in message queue
3. Client side of SMTP at mail server opens TCP connection with Bob’s mail server
4. SMTP client sends Alice’s message over the TCP connection
5. Bob’s mail server places the message in Bob’s mailbox
6. Bob invokes his user agent to read message



# SMTP interaction on protocol level

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

# SMTP vs HTTP

## HTTP

- ▶ Client Pull
- ▶ ASCII commands
- ▶ ASCII status codes
- ▶ Each object encapsulated in its own response message
- ▶ Non-persistent & persistent connections

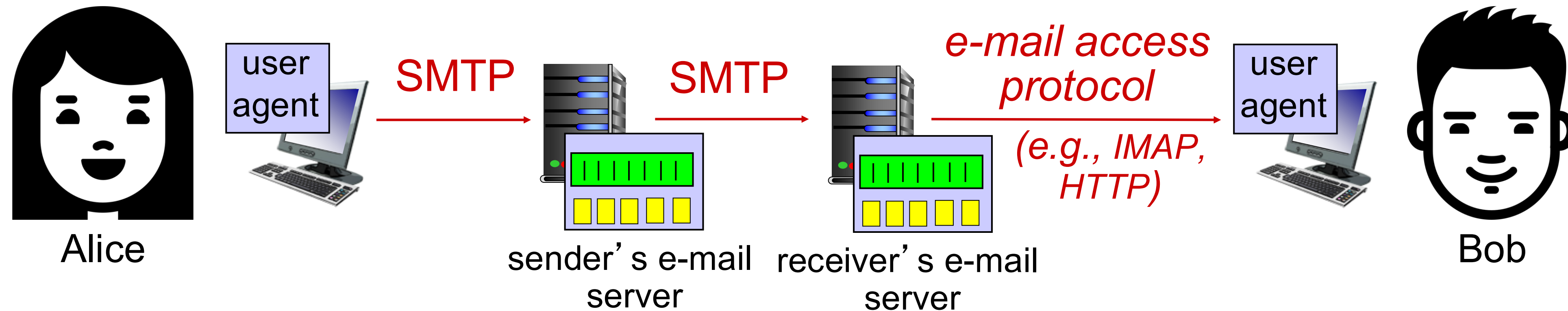
## SMTP

- ▶ Client Push
- ▶ ASCII commands
- ▶ ASCII status codes
- ▶ Multiple objects sent in multipart message
- ▶ Persistent connections



This is super versatile, but will bite us in the a\*\* when it comes to security

# Retrieving email: Mail access protocols



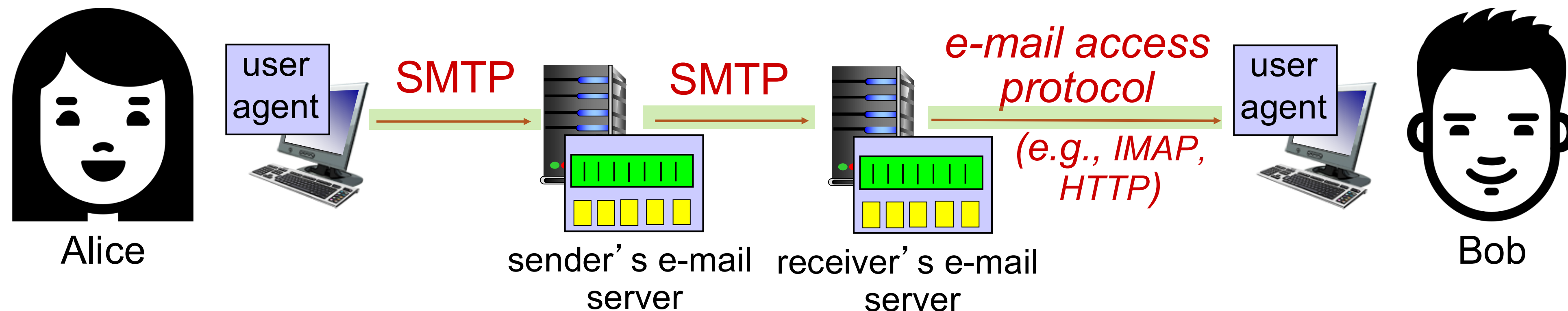
- ▶ **SMTP**: delivery/storage of e-mail messages to receiver's server
- ▶ **Mail access protocol**: retrieval from server
  - ▶ **IMAP**: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- ▶ **HTTP**: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

# Securing E-mail

- ▶ In this lecture we only talk about the basics on how to achieve
  - ▶ Confidentiality of message content
  - ▶ Authenticity of sender
- ▶ Two options
  - ▶ Transport Layer Security
  - ▶ End-to-end security

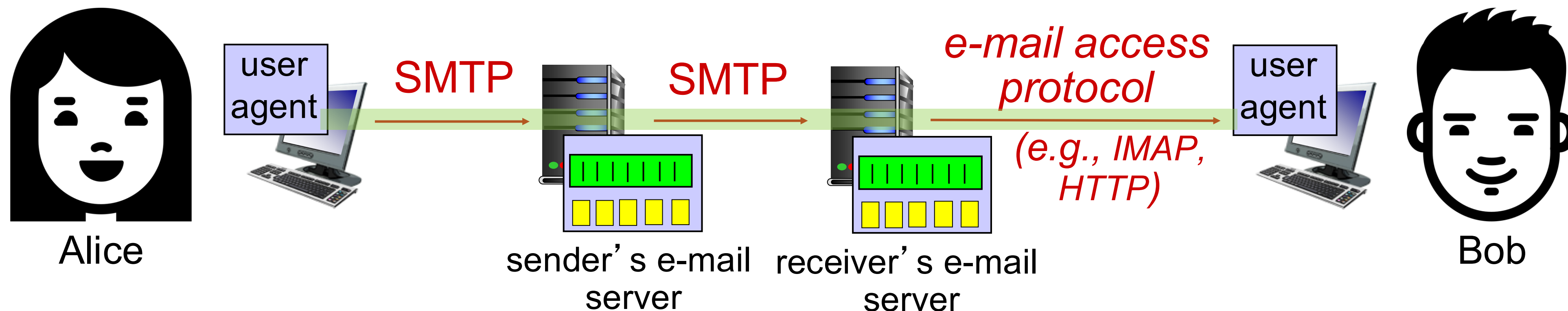
# Securing E-mail – Option 1: Transport Layer Security

- ▶ Assumptions
  - ▶ Clients and servers trusted/not compromised
  - ▶ Network untrusted/can be accessed by attacker
- ▶ Protocols encapsulated in secure connections on transport layer (TLS)
- ▶ Advantage: easier, since it only needs to be deployed by servers
- ▶ Disadvantage: servers must be trusted
- ▶ We talk more about TLS in the transport layer lecture

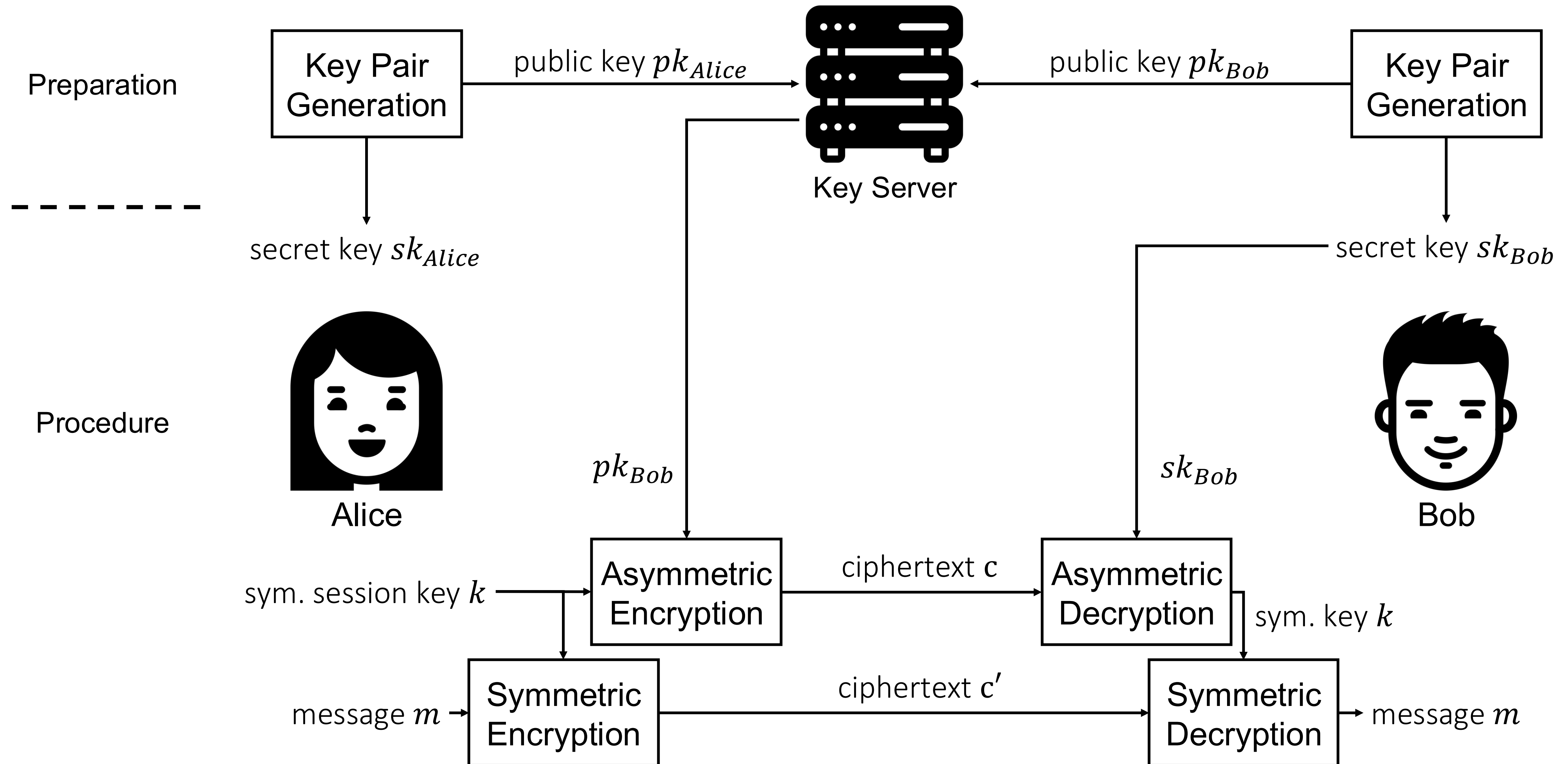


# Securing E-mail – Option 2: End-to-end Security

- ▶ Assumptions
  - ▶ Clients trusted/not compromised
  - ▶ Network and servers untrusted/can be accessed by attacker
- ▶ Message body encrypted/decrypted & signed/verified by user agents
- ▶ Advantage: Strictly more secure
- ▶ Disadvantage: More complicated for users

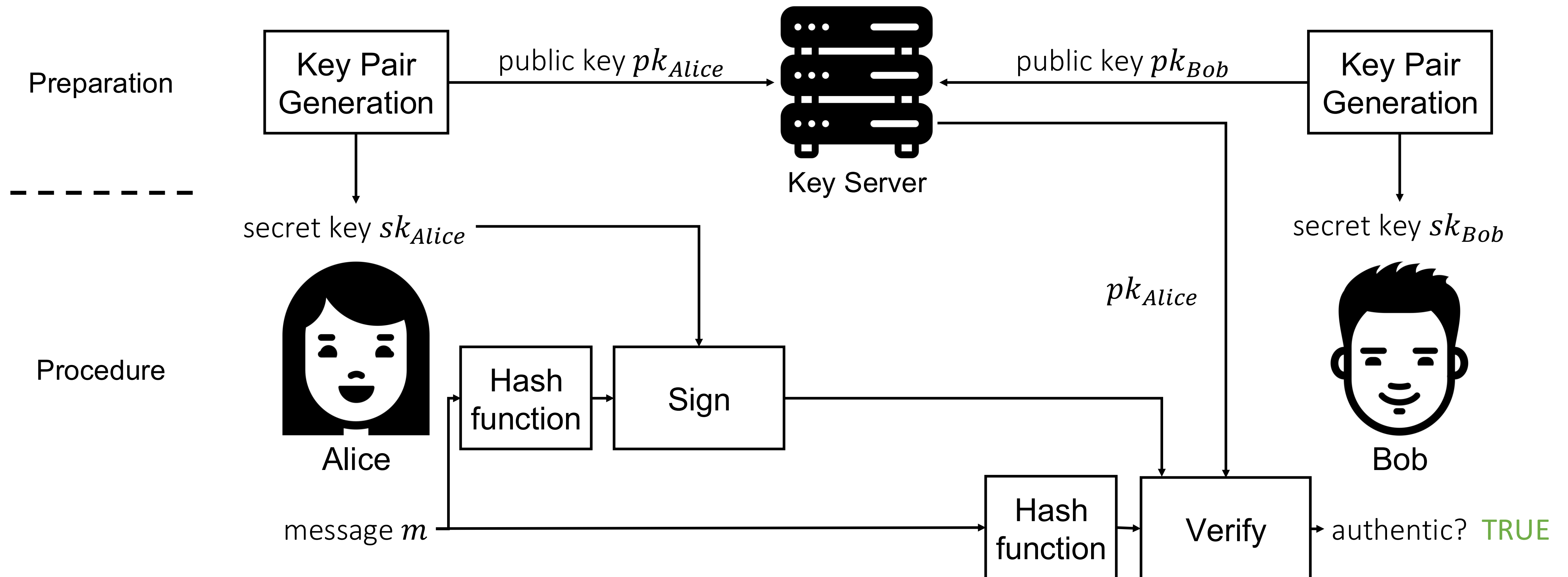


# Securing E-mail – Recap Hybrid Encryption



# Securing E-mail – Recap Digital Signatures

- Provide authenticity of data through public key cryptography
- Use RSA or ECDSA (Elliptic Curve Digital Signature Algorithm)



# Securing E-mail – Option 2: End-to-end Security

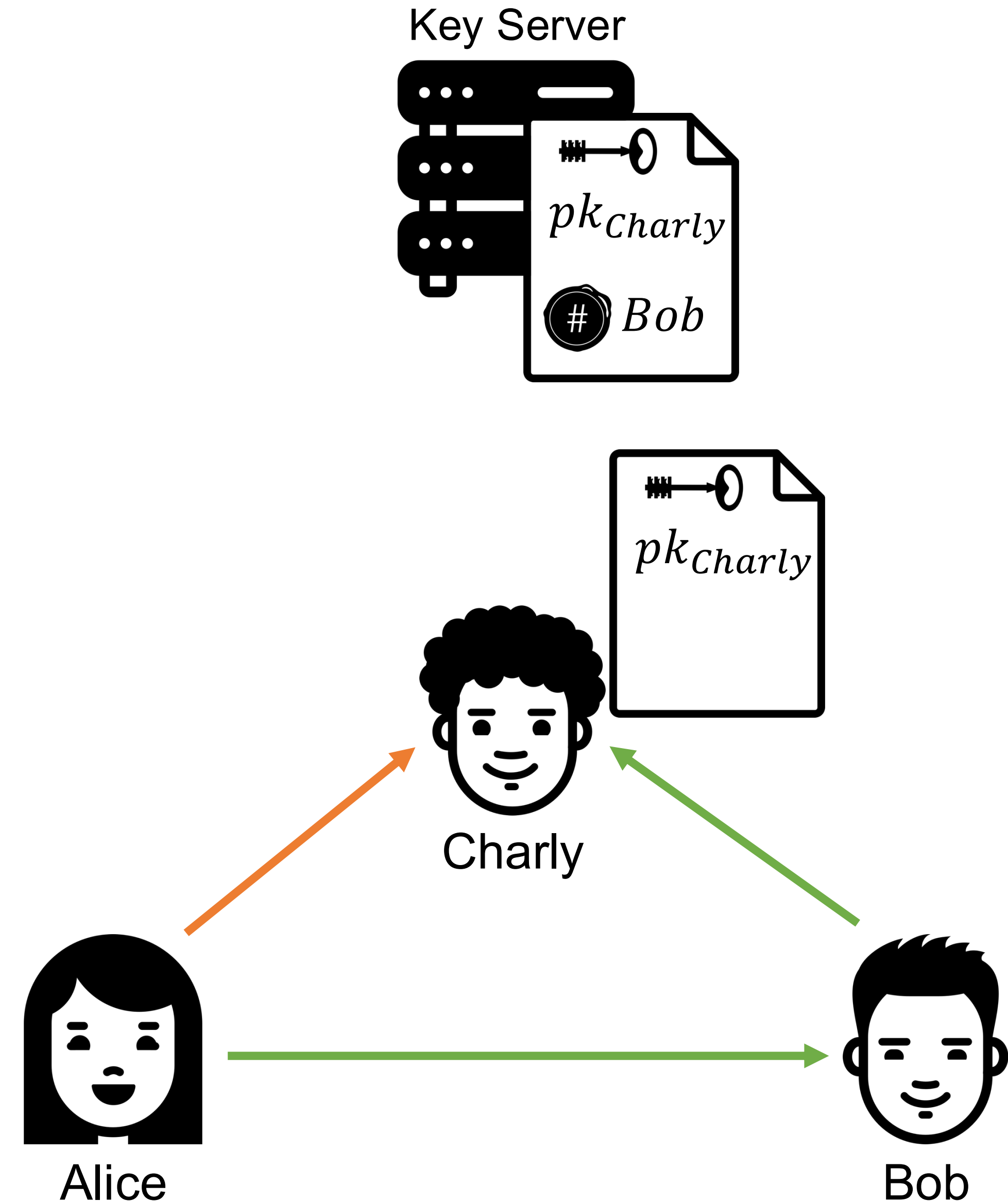
- ▶ Based on hybrid encryption and digital signatures
- ▶ Two standards for end-to-end security
  - ▶ OpenPGP
  - ▶ S/MIME
- ▶ Different principles of trust behind them
- ▶ Both come with trade-offs
- ▶ We will look at the basic concepts of both of them

# Securing E-mail – OpenPGP

- ▶ Technically the older standard, due to predecessor PGP (1991)
  - ▶ OpenPGP specified in RFC 4880 with many extensions
  - ▶ However, less pervasive
- ▶ Basic principle: people create their own keys and publish them wherever they see fit
  - ▶ Usually at keyservers, e.g. <https://keys.openpgp.org>
- ▶ Everyone can upload keys... How do you know which is the one?
- ▶ Based on the web-of-trust principle
  - ▶ People can attest that they know someone and have verified that a certain key belongs to them
  - ▶ When you trust at least one of the signatures, you can trust the key

# Web-of-Trust Example

- ▶ Assume Alice wants to send a message to Charly ...
- ▶ ... but Alice does not know Charly directly.
- ▶ However, Alice knows Bob ...
- ▶ ... and Bob knows Charly ...
- ▶ ... and Charly has uploaded his public key to a key server.
- ▶ And Bob has signed Charly's key.
- ▶ So Alice can download the key, verify the signature and be confident to write the email.

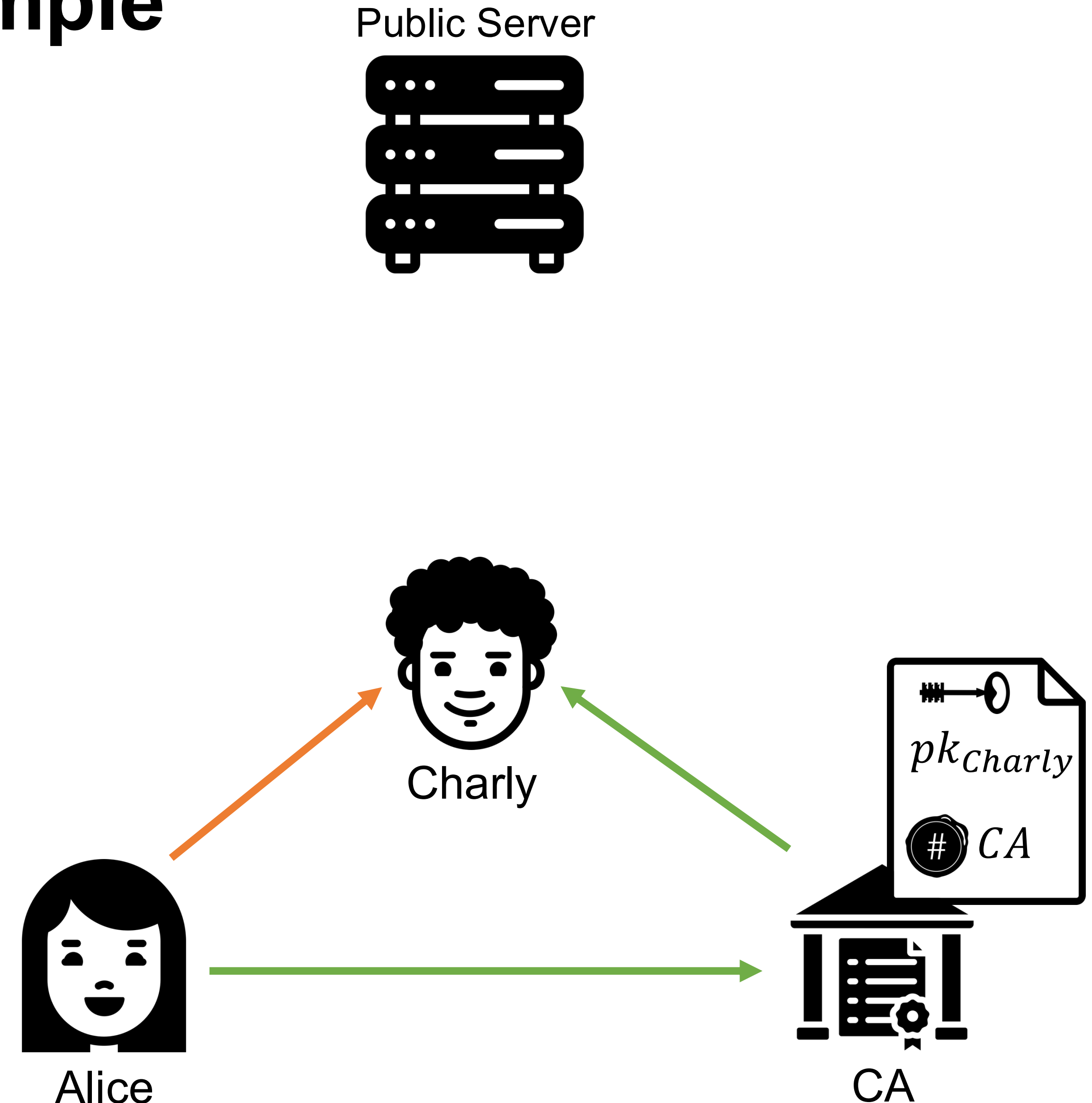


# Securing E-mail – S/MIME

- ▶ Secure / Multipurpose Internet Mail Extensions
- ▶ Specified in RFC 1847, again with extensions
- ▶ Basic principle: Public Key Infrastructure (PKI)
  - ▶ Certificate Authorities (CAs) create and sign keys
  - ▶ Everyone trusts the CAs and therefore can verify the authenticity of the keys

# Public Key Infrastructure Example

- ▶ Assume Alice wants to send a message to Charly ...
- ▶ ... but Alice does not know Charly directly.
- ▶ However, Alice trusts the Certificate Authority ...
- ▶ ... and Charly has gotten a signed key from the CA ...
- ▶ ... which he has uploaded to a public server.
- ▶ So Alice can download the key, verify the signature and be confident to write the email.



Learning Goal 63475C

# What is DNS?

# How to find the right host to contact?

- ▶ People: many identifiers:
  - ▶ CPR#, name, passport#, ...
- ▶ Internet hosts, routers:
  - ▶ IP address (32 bit) - used for addressing datagrams
- ▶ “Name”, e.g., imada.sdu.dk - used by humans
- ▶ Q: How to map between IP address and name, and vice versa?

# The Domain Name System (DNS)

- ▶ Distributed database implemented in hierarchy of many name servers
- ▶ Application-layer protocol: hosts, DNS servers communicate to resolve names (address/name translation)
  - ▶ Note: core Internet function, implemented as application-layer protocol
  - ▶ Complexity at network's "edge"
- ▶ DNS services
  - ▶ Hostname-to-IP-address translation
  - ▶ Host aliasing
  - ▶ Canonical, alias names
  - ▶ Mail server aliasing
  - ▶ Load distribution
  - ▶ Replicated web servers: many IP addresses correspond to one name

# Distributed vs Centralised DNS

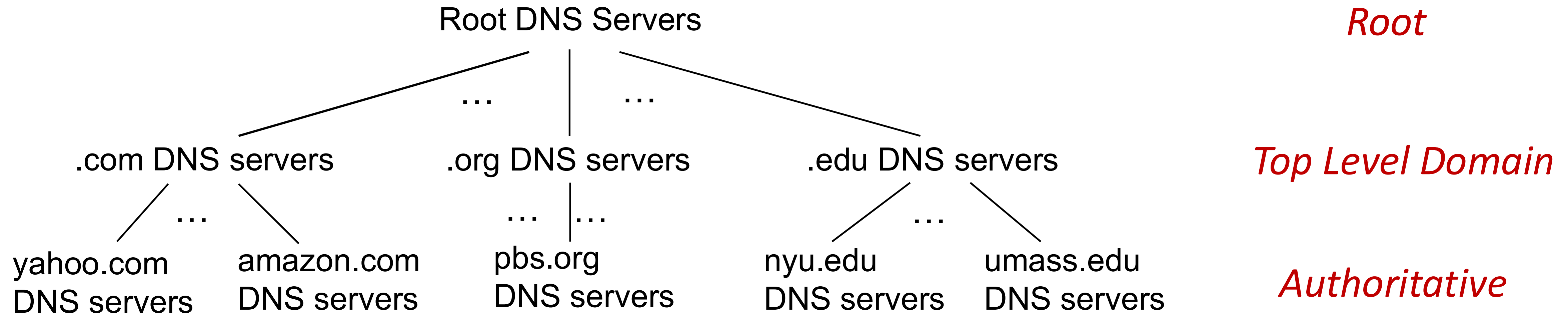
- ▶ Distributed database implemented in hierarchy of many name servers
- ▶ Q: Why distributed not centralized DNS?
  - ▶ Single point of failure
  - ▶ Traffic volume
  - ▶ Distant centralized database
  - ▶ Maintenance
  - ▶ Does not scale (Akamai 2.2T DNS queries/day)

# Thinking about the DNS

- ▶ Humongous distributed database
  - ▶ Billion records, each simple
- ▶ Handles many trillions of queries/day
  - ▶ Many more reads than writes
  - ▶ Performance matters: almost every Internet transaction interacts with DNS - msec count!
- ▶ Organizationally, physically decentralized
  - ▶ Millions of different organizations responsible for their records
- ▶ “Bulletproof”: reliability, security



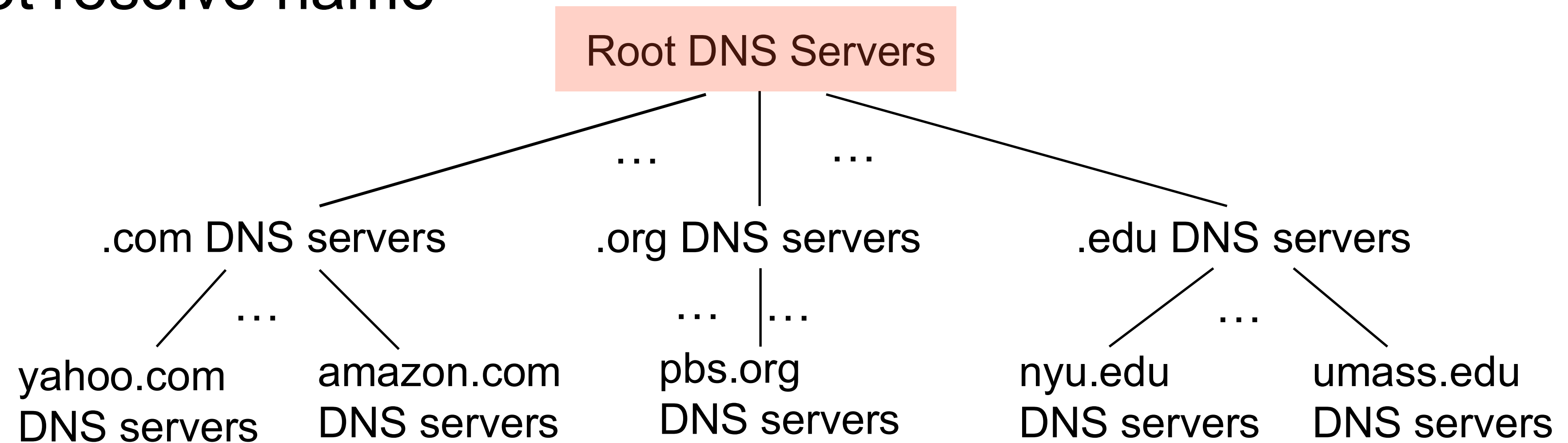
# DNS – A Distributed, Hierarchical Database



- ▶ Client wants IP address for `www.amazon.com`; 1st approximation
  - ▶ Client queries root server to find `.com` DNS server
  - ▶ Client queries `.com` DNS server to get `amazon.com` DNS server
  - ▶ Client queries `amazon.com` DNS server to get IP address for `www.amazon.com`

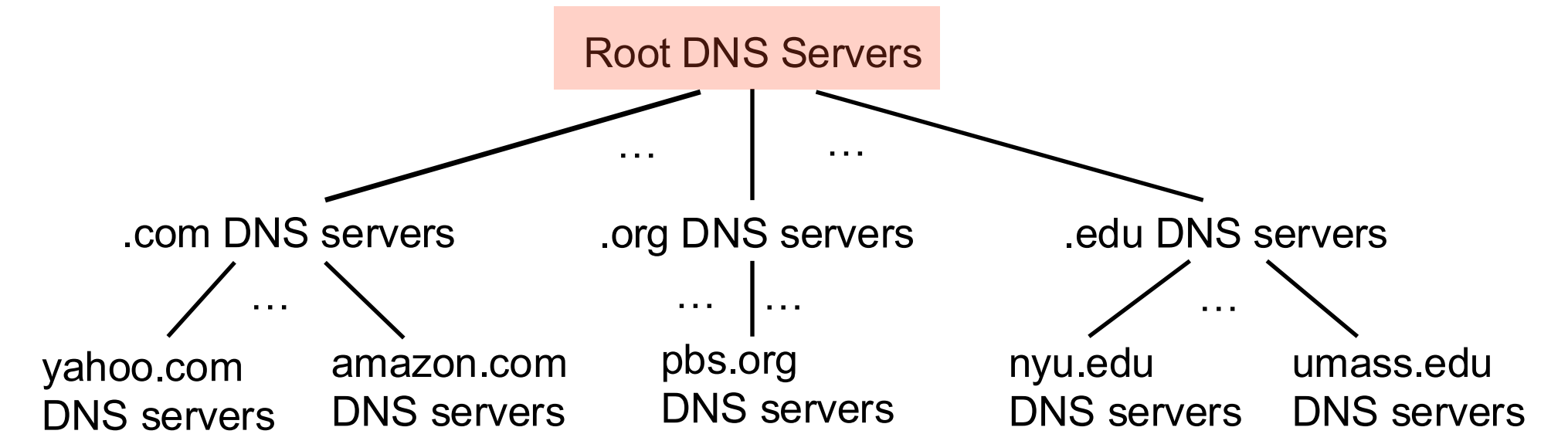
# DNS – Root Servers

- Official, contact-of-last-resort by name servers that can not resolve name

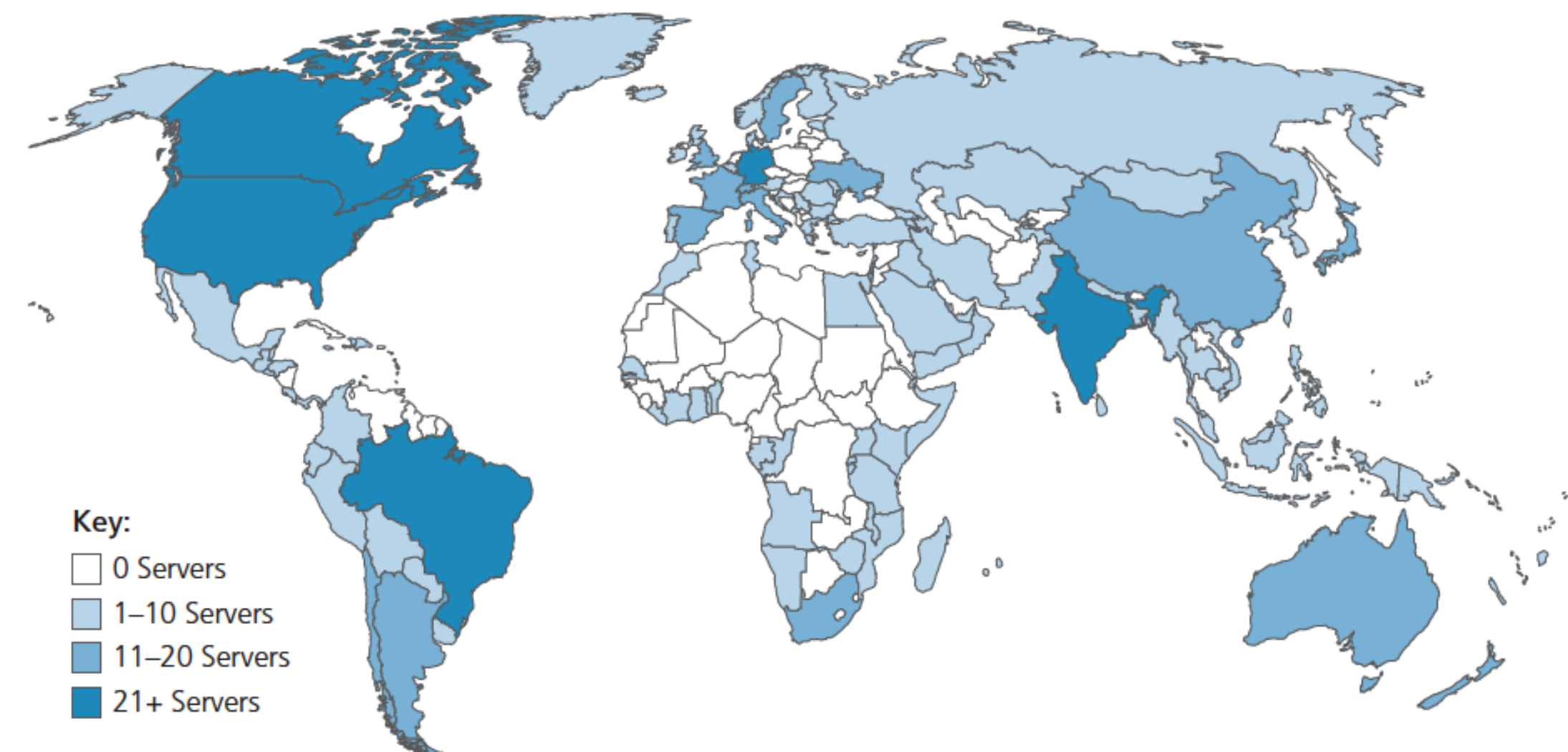


# DNS – Root Servers

- ▶ Official, contact-of-last-resort by name servers that can not resolve name
- ▶ Incredibly important Internet function
- ▶ Internet couldn't function without it!
- ▶ ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

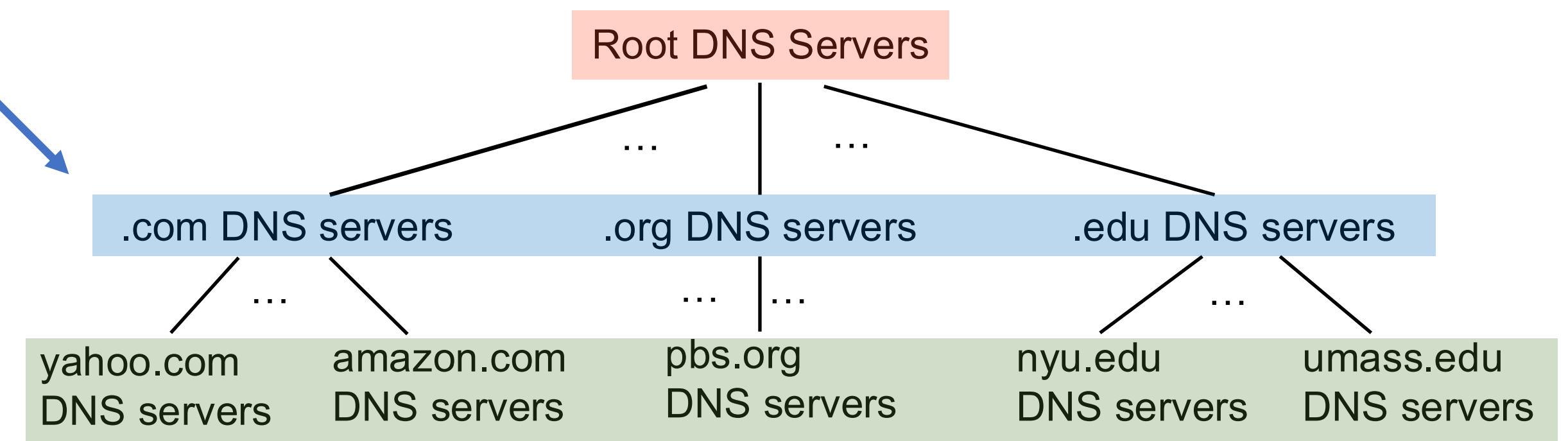


13 logical root name “servers”  
worldwide each “server” replicated  
many times (~200 servers in US)



# DNS – Top-Level Domain & Authoritative Servers

- ▶ **Top-Level Domain (TLD) servers:** Responsible for .com, .org, .net, .edu, etc. and all top-level country domains, e.g.: dk., .cn, .fr, .ca, .jp



- ▶ **Authoritative DNS servers**

- ▶ Organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- ▶ Can be maintained by organization or service provider

# Local DNS Name Servers

- ▶ When host makes DNS query, it is sent to its local DNS server
  - ▶ Local DNS server returns reply, answering:
    - ▶ From its local cache of recent name-to-address translation pairs (possibly out of date!)
    - ▶ Forwarding request into DNS hierarchy for resolution
  - ▶ Each ISP has local DNS name server; to find yours:
    - ▶ MacOS: % scutil --dns
    - ▶ Windows: >ipconfig /all
- ▶ Local DNS server doesn't strictly belong to hierarchy

# DNS Name Resolution – Iterated Query

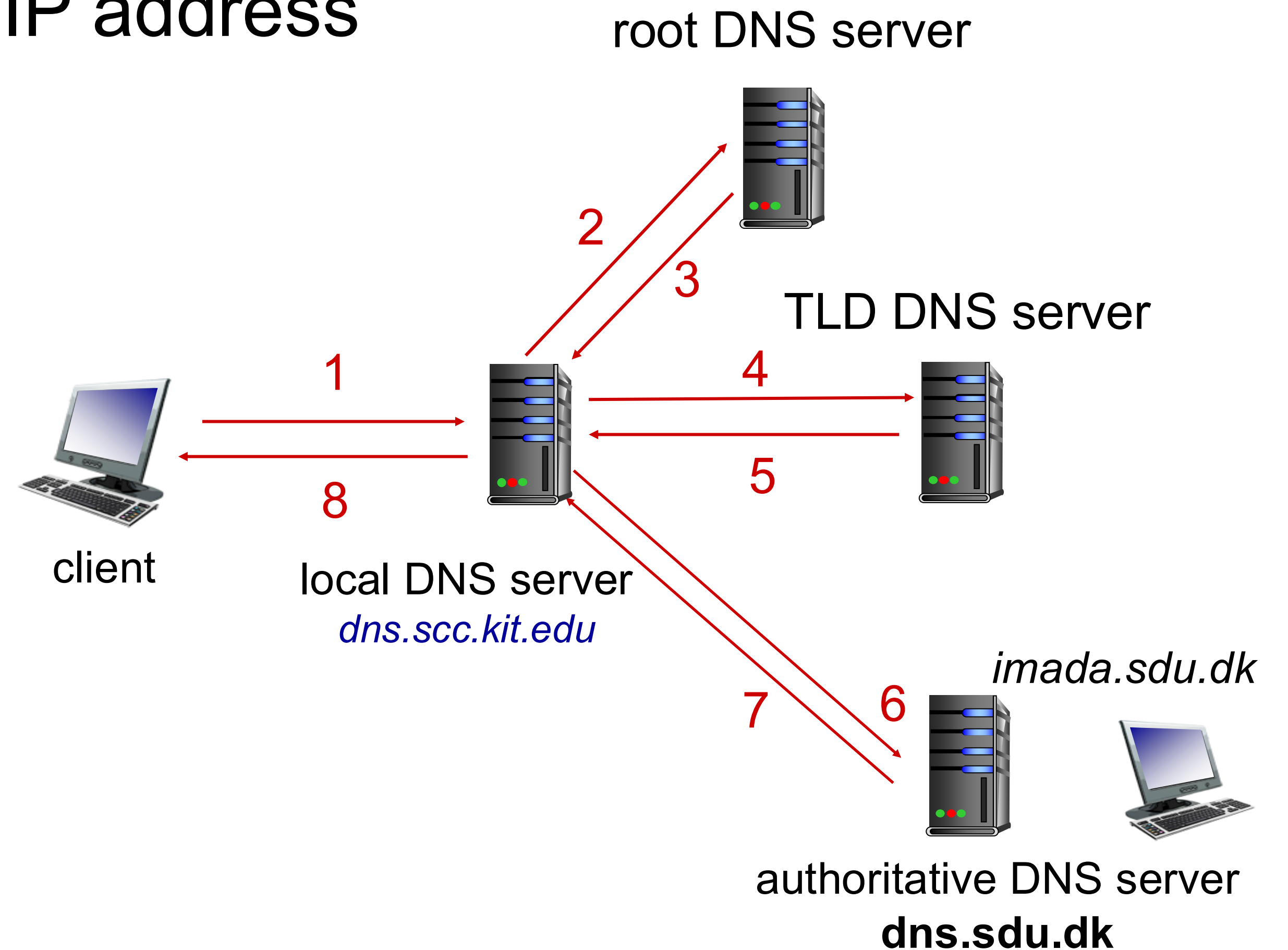
- ▶ Example: client at ITU on wants IP address

for `imada.sdu.dk`

- ▶ Iterated query

- ▶ Contacted server replies with name of server to contact

- ▶ “I don’t know this name, but ask this server”



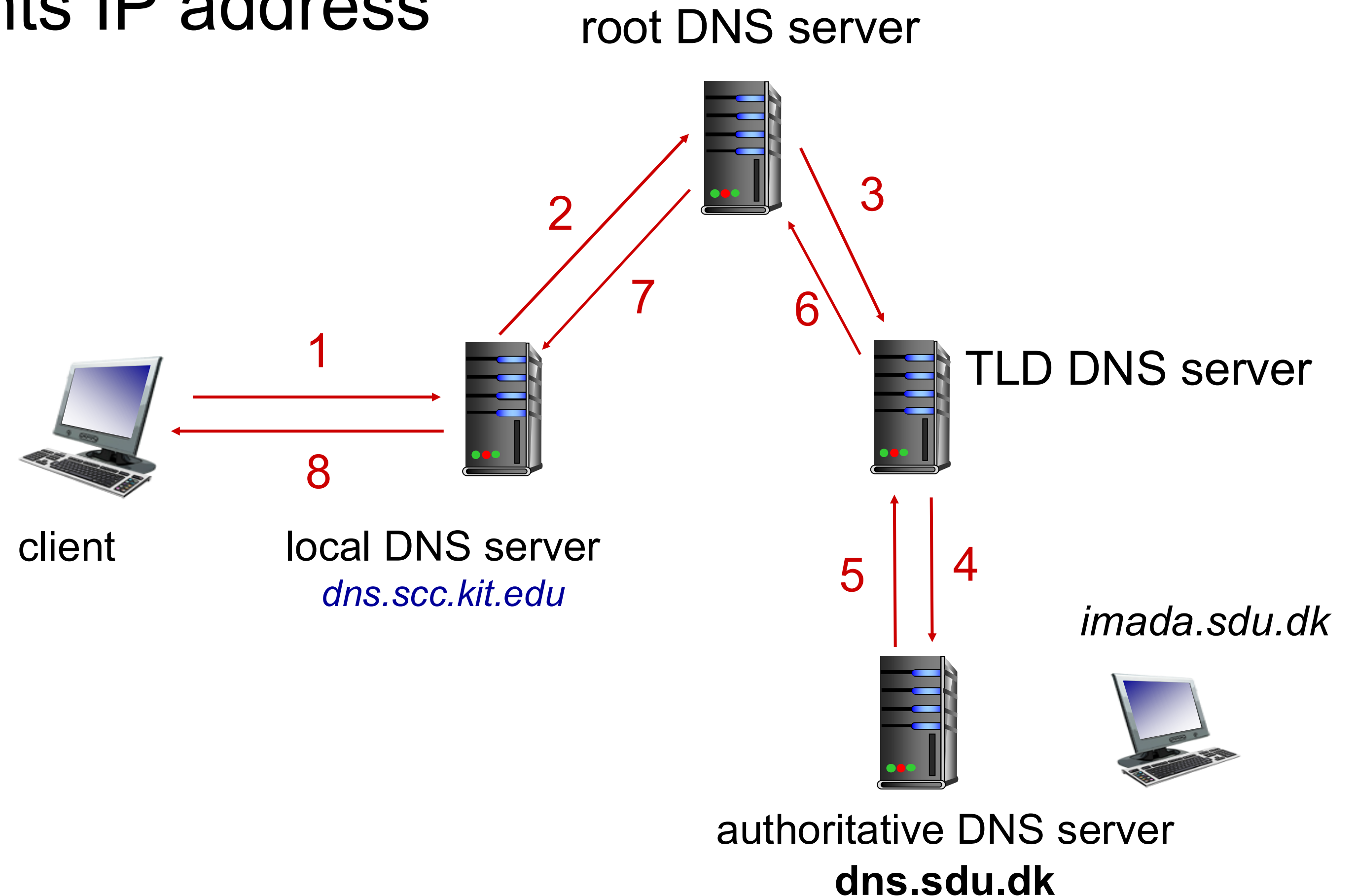
# DNS Name Resolution – Recursive Query

- ▶ Example: client at KIT on wants IP address

for `imada.sdu.dk`

- ▶ Recursive query

- ▶ Puts burden of name resolution on contacted name server
- ▶ Heavy load at upper levels of hierarchy?
- ▶ Allows for more effective caching



# Caching DNS Information

- ▶ Once (any) name server learns mapping, it caches mapping, and immediately returns a cached mapping in response to a query
  - ▶ Caching improves response time
  - ▶ Cache entries timeout (disappear) after some time (TTL)
  - ▶ TLD servers typically cached in local name servers
- ▶ Cached entries may be out-of-date
  - ▶ If named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - ▶ Best-effort name-to-address translation!

# DNS records

**DNS:** distributed database storing resource records  
(RR)      RR format: (name, value, type, ttl)

## type=A / AAAA

- ▶ name is hostname
- ▶ value is IP address

## type=NS

- ▶ name is domain (e.g., foo.com)
- ▶ value is hostname of authoritative name server for this domain

## type=CNAME

- ▶ name is alias name for some “canonical” (the real) name
- ▶ www.ibm.com is really east.backup2.ibm.com
- ▶ value is canonical name

## type=MX

- ▶ value name of SMTP mail server associated with name

# DNS Protocol Messages

DNS *query* and *reply* messages, both have same *format*:

## Message Header:

- ▶ Identification: 16 bit # for query, reply to query uses same #
- ▶ Flags:
  - ▶ Query or reply
  - ▶ Recursion desired
  - ▶ Recursion available
  - ▶ Reply is authoritative

|                                     |                  |
|-------------------------------------|------------------|
| ← 2 bytes → ← 2 bytes →             |                  |
| identification                      | flags            |
| # questions                         | # answer RRs     |
| # authority RRs                     | # additional RRs |
| questions (variable # of questions) |                  |
| answers (variable # of RRs)         |                  |
| authority (variable # of RRs)       |                  |
| additional info (variable # of RRs) |                  |

# DNS Protocol Messages

DNS *query* and *reply* messages, both have same *format*:

← 2 bytes      ← 2 bytes      →

|                                     |                  |
|-------------------------------------|------------------|
| identification                      | flags            |
| # questions                         | # answer RRs     |
| # authority RRs                     | # additional RRs |
| questions (variable # of questions) |                  |
| answers (variable # of RRs)         |                  |
| authority (variable # of RRs)       |                  |
| additional info (variable # of RRs) |                  |

name, type fields for a query

RRs in response to query

records for authoritative servers

additional “ helpful” info that may  
be used

# Getting your info into the DNS

- ▶ Example: new startup “Network Utopia”
- ▶ Register name networkutopia.com at DNS registrar
- ▶ Provide names, IP addresses of authoritative name server (primary and secondary)
- ▶ Registrar inserts NS, A RRs into .com TLD server:
  - ▶ (`networkutopia.com`, `dns1.networkutopia.com`, NS)
  - ▶ (`dns1.networkutopia.com`, `212.212.212.1`, A)
- ▶ Create authoritative server locally with IP address `212.212.212.1`
  - ▶ Type A record for `www.networkutopia.com`
  - ▶ Type MX record for `networkutopia.com`

# DNS Security

## DDoS attacks

- ▶ Bombard root servers with traffic
  - ▶ Not successful to date
  - ▶ Traffic filtering
  - ▶ Local DNS servers cache IPs of TLD servers, allowing root server bypass
- ▶ Bombard TLD servers
  - ▶ Potentially more dangerous
- ▶ Use DNS requests with spoofed IP to multiply bandwidth of attack
  - ▶ This technique has lead to some of the largest DDoS attacks so far

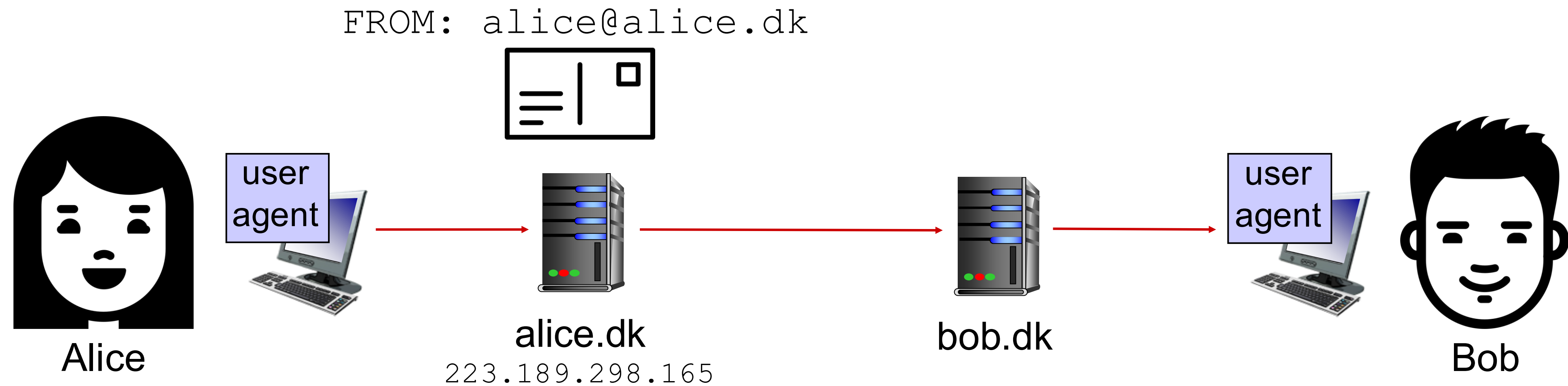
## Spoofing Attacks

- ▶ Intercept DNS queries, returning bogus replies
  - ▶ DNS cache poisoning
  - ▶ RFC 4033: DNSSEC authentication services

Learning Goal 709843

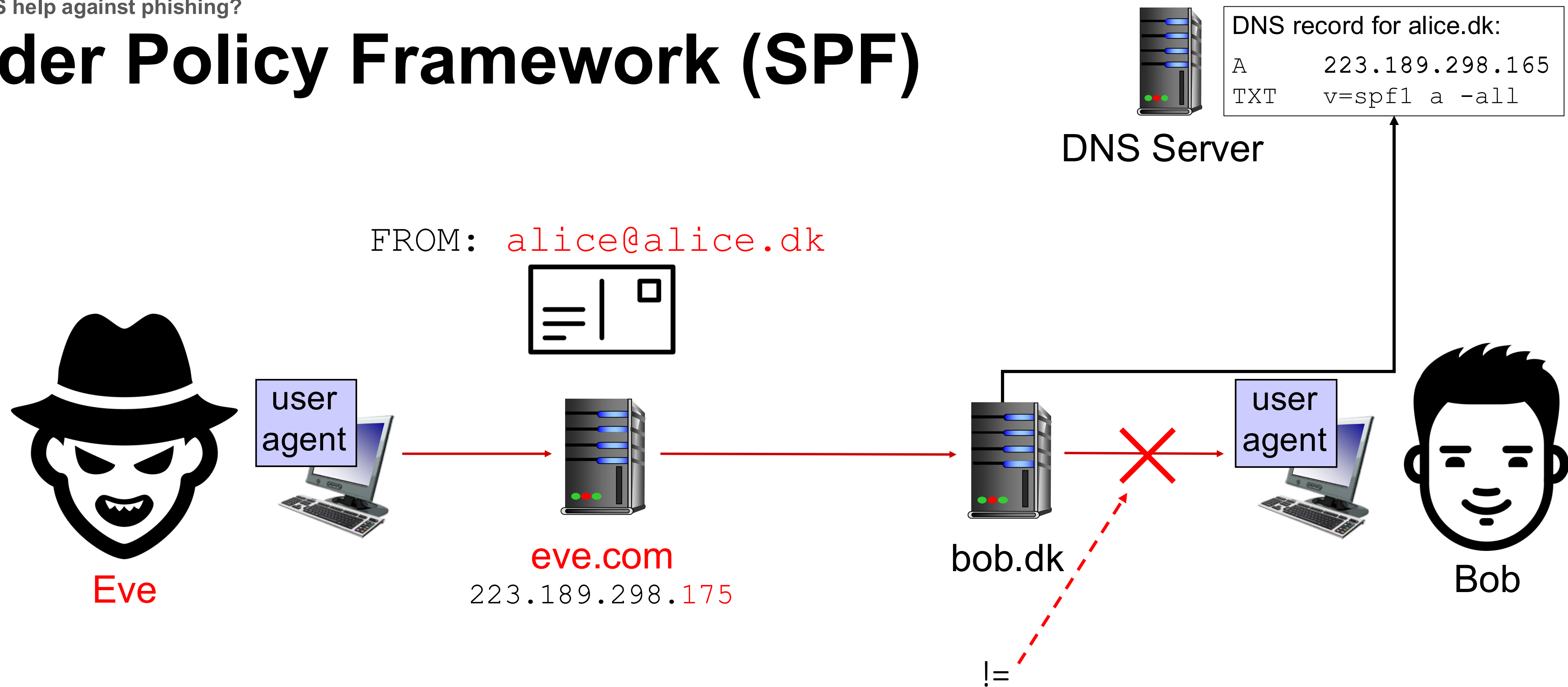
# How can DNS help against phishing?

# Sender Spoofing



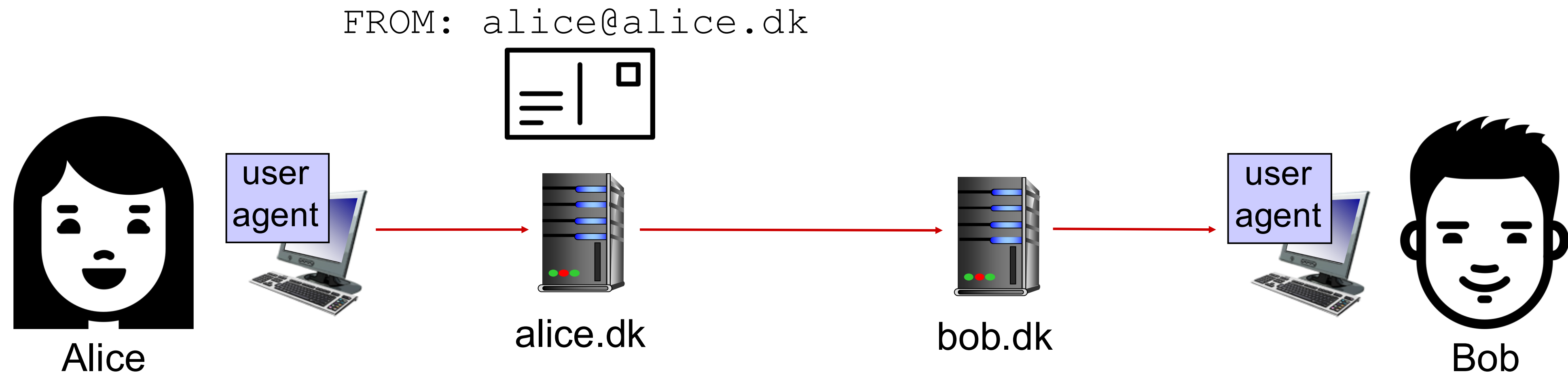
- ▶ Example: Alice wants to send an email to Bob
- ▶ Assumption: Alice and Bob do not use end-to-end secure email

# Sender Policy Framework (SPF)



- ▶ Example: Eve wants to impersonate Alice
  - ▶ Uses own server eve.com to send email with spoofed sender `alice@alice.dk`
  - ▶ Idea: Use DNS record which servers are allowed to send for `alice.dk`

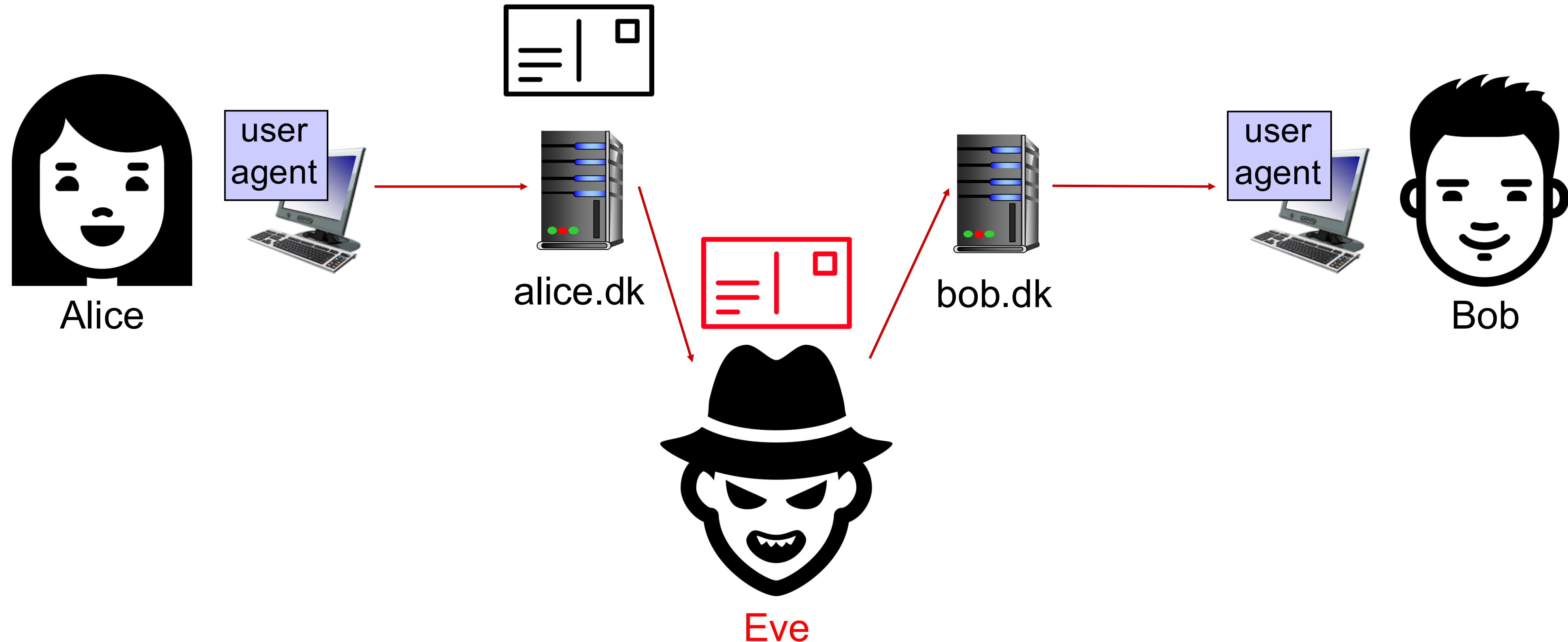
# Message Manipulation



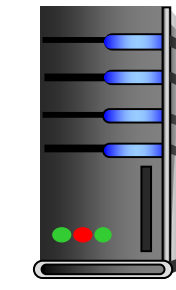
- ▶ Example: Alice wants to send an email to Bob
- ▶ Assumption: Alice and Bob do not use end-to-end secure email

# Message Manipulation

- ▶ Example: Eve wants to modify Alice's message
  - ▶ Intercepts message and manipulates it in transit
- ▶ Idea: Use digital signatures between servers to protect message



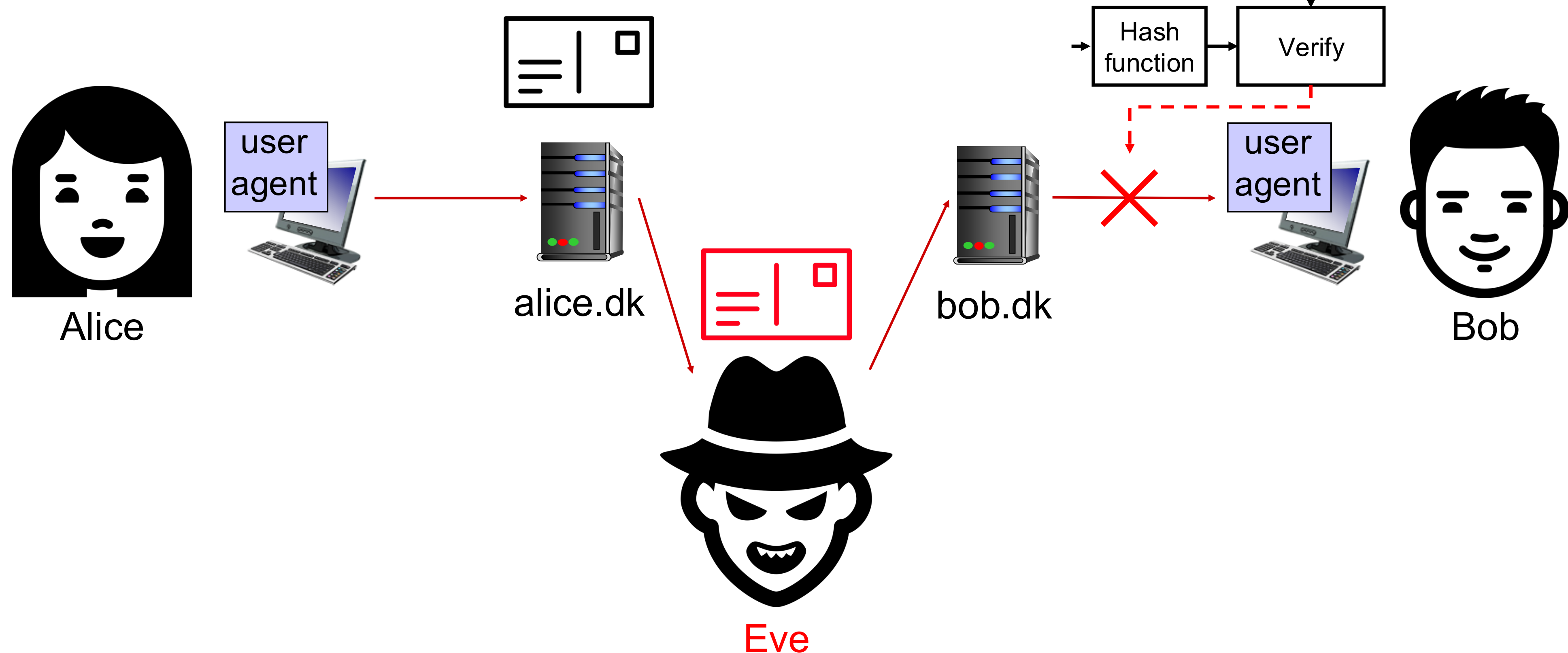
# DomainKeys Identified Mail (DKIM)



DNS Server

DNS record for `_domainkey.alice.dk`:

```
TXT    k=rsa; t=s; p=MIGfMA0GCSqGSIb3
       DQEBAQUAA4GNADCBiQKBgQDDmzRmJR
       QxLEuyYiyMg4suA2SyMwR5MGH...
```



# DMARC

- ▶ Domain-based Message Authentication, Reporting and Conformance
- ▶ Combines SPF and DKIM protections
- ▶ Published as DNS TXT RR
- ▶ Includes
  - ▶ Policies on how to handle messages in case of verification failures
    - ▶ Reject the message
    - ▶ Quarantine the message
  - ▶ Generating reports for senders
- ▶ Recommended standard for email security

DNS record for \_dmarc.alice.dk:

```
TXT    v=DMARC1;p=none;sp=quarantine;  
       pct=100;rua=mailto:dmarcreport  
       s@alice.dk;
```

# Learning Goals

- ▶ Get an understanding of network applications are ✓
  - ▶ Types of network applications
  - ▶ Client-Server Paradigm
  - ▶ Peer-to-peer architectures
  - ▶ Sockets & Process Communication
- ▶ Understand what a application layer protocol is ✓
  - ▶ Properties
  - ▶ Examples

# Learning Goals

- ▶ Get an understanding of HTTP ✓
  - ▶ Basic principles
  - ▶ Persistent & Non-persistent
  - ▶ Requests, responses & their formats
  - ▶ Maintaining state in the stateless HTTP
  - ▶ User tracking with cookies
  - ▶ HTTP/2
  - ▶ Web caching and HTTP performance

# Learning Goals

- ▶ Get an first understanding of e-mail ✓
  - ▶ Terminology
  - ▶ The SMTP protocol
  - ▶ Retrieving e-mails
  - ▶ Securing emails with OpenPGP & S/MIME
- ▶ Get an understanding of the Domain Name System ✓
  - ▶ The DNS hierarchy and types of servers
  - ▶ Iterated & recursive queries
  - ▶ Caching
  - ▶ DNS protocol messages
  - ▶ DNS Security

# Learning Goals

- ▶ Get an understanding of how DNS can help against phishing ✓
  - ▶ Sender Policy Framework
  - ▶ DomainKeys Identified Mail
  - ▶ Domain-based Message Authentication, Reporting and Conformance